UNIT 1 PYTHON PRIMER

Structure

1.0 Introduction
1.1 Objectives
1.2 Python Language Overview
1.3 Data Types in Python
1.4 Classes and Objects in Python
1.5 Aliasing in Python
1.6 Expressions in Python
1.7 Control Flow
1.8 Looping / Repetition
1.9 Functions
1.10 Built-in functions
1.11 Taking Multiple inputs from Console
1.12 Exception Handling
1.13 Built-in Exceptions in Python
1.14 Iterators and Generators
1.15 Generators in Python
1.16 Additional Python Conveniences
1.17 Scopes and Namespaces
1.18 Summary
1.19 Further Readings

1.0 INTRODUCTION

Any data to be stored or retrieved from our computer requires usage of data structures. Creating and using these data structures can be done with plethora of high level programming languages like C, C++, Java or Python. In recent years, Python has become a language of choice since it has simple syntax, easier code readability, user friendly and can be used with wide variety of applications. It has extensive library, large active user community, cross-platform compatibility and beginner-friendly design philosophy. To understand its acceptability, for web and software development, Python is behind many websites and applications. For understand the hidden data patterns, Data analysts use Python's libraries to process and visualize huge datasets. In the fields of machine learning and artificial intelligence, Python is vastly used. Python's journey began with its first release, Python 0.9.0, in 1991 although it has undergone significant enhancements, with Python 2.0 introducing list comprehensions and garbage collection. A major transition occurred with the introduction of Python 3.0 in 2008 that made it more versatile.

Most of the high level languages are compiled but a few are interpreted. In a compiled language, the high level code is translated into machine code or executable code with the aid of a compiler before execution. On the other hand, interpreter reads each line of code, interprets it into machine instructions, and executes it straight away. There isn't any need of separate compilation. Python code is interpreted language through Python interpreter. The interpreter receives a command, evaluates that command, and reports the result of the command. The source code in Python is conventionally stored in a file named with the .py suffix like hello.py. These programs are executed by the interpreter.

1.1 OBJECTIVES

The purpose of this unit is to introduce and fly you across the world of Python to the extent possible.

After studying this unit, you should be able to understand :

- Syntax of Python
- Different operators in Python

It is also strongly recommended that, in parallel, register to the free courses of Python which are given in references at the end of this unit.

1.2 PYTHON LANGUAGE OVERVIEW

Python is a dynamic, interpreted or a byte code-compiled language. This means that prototyping can be very quick while testing the code. As you code in Python, you shall look at it like procedural at times or object-oriented. One doesnot have to bother about the type declarations of variables, parameters, functions, or methods in code you write. Python interpreter tracks the types of all values at runtime and flags code that does not make sense when it runs. Python code works on variety of platforms like Windows, Mac, Linux, Raspberry Pi, etc. It has a simple syntax quite similar to the English language. It requires lesser lines as compared to other high level programming languages because of its rich library.

1.2.1 How to install Python on Windows

To install Python on Windows systems, one can visit the python.org download page and download the current Python 3.X.X version. Now, Run the Python installer and accept all the defaults that are displayed. This will install Python in the root directory. Now the complete Python is installed and one can open the command prompt from:

Accessories > Command Prompt, or

type cmd into the run dialog

Your First Python program

This program prints Welcome to Python

Print('Welcome to Python')

Note that we use the built-in print() function to print the string Welcome to Python on the screen.

1.3 DATA TYPES IN PYTHON

Data types define the type of a variable that represents the type of data that can be stored in a variable and the kind of operations that can be done on it. The table below lists the builtin data types in Python.

Data Type	Description
Numeric	int, float, complex
Sequence	list, tuple range
Mapping	dict
Set	set, frozenset
Text	str
Binary	bytes, bytearray, memoryview

Numeric Types:

int: Used to represents whole numbers (e.g.: 2, -15, 0).

float: Used to represents decimal numbers (e.g.: 2.14, -0.8, 8.0).

complex: Used to represents complex numbers in the form a + bk, where a and b are floats and k is the imaginary unit (e.g.: 6 + 8k, -1k).

Sequence Types:

str: Used to represents text as a sequence of characters (e.g.: "good", 'world', "987").

list: Used to represents an ordered, mutable (modifiable) sequence of items, which can be of different types (e.g., \[1, "gold", 2.77]).

tuple: Used to represents an ordered, immutable (un-modifiable) sequence of items (e.g.: [10, "amul", 3.14)).

range: Used to represents a sequence of numbers (e.g.: range(3) generates numbers from 0 to 2).

Mapping Type:

dict: Used to represents a collection of key-value pairs, where keys are unique and immutable (e.g.: {"name": "Karan", "age": 27}).

Set Types:

set: Used to represents an unordered collection of unique items (e.g., {2,4,8}).

frozenset: Used to represents an immutable version of a set.

Boolean Type:

bool: Used to represents truth values that can have two states True or False.

Binary Types:

bytes: Used to represents immutable sequence of bytes.

bytearray: Used to represents mutable sequence of bytes.

memoryview: Provides us with a view of an object's memory.

1.4 CLASSES AND OBJECTS IN PYTHON

Python is an object-oriented language and classes form the basis for all its data types. Classes are a means of bringing together data and functionality together. Each class encapsulates data and behaviour (methods) into a single entity. This structure allows you to model real-world objects and create organized, reusable code. A class in Python serves as a template for creating objects, which are instances of the class or instantiated from the class. We use classes when we need to encapsulate related data and functions, making the code modular and easier to manage. After defining a class, one can create multiple objects that share the same attributes and methods, while all these objects have their own unique state.

Example:

class car:

name = “”

Colour = “”

An object is an instance of a class. Creating a new instance of a class is known as instantiation. Hence Car is a class and now we can create objects like car1, car2 etc from this class as under:

Class car:

name = “BMW”

Colour = “white”

1.4.1 Identifiers, Objects, and the Assignment Statement

Identifiers

In Python, an identifier is a name given to a programming element, such as a variable, function, class, or module. It allows programmers to refer to these elements within their code. On the other hand, a variable, is an entity whose value can changea over time. It denotes a memory location where a value can be stored. Naming rules for identifiers require these to begin with a letter or underscore, followed by letters, underscores, or digits. Additionally, it’s essential to avoid using reserved words as identifiers. The first letter of an identifier cannot be a digit. These are case sensitive and hence "variable" and "Variable. Are two distinct identifiers". Whitespaces are not allowed in the identifiers. The scope of identifiers can be either local or global, dictating their accessibility within the program.

Reserved Words

Python 3 has 33 while Python 2 has 30 reserved words. The reserved word print was removed from Python 2 keywords and added as a built-in Python function.

False	def	if	raise
None	del	import	return
True	elif	in	try
and	else	is	while
as	except	lambda	with
assert	finally	nonlocal	yield
break	for	not	
class	form	or	
continue	global	pass	

Table 1: List of Python reserved words in Python that can’t be used as identifiers.

1.5 ALIASING IN PYTHON

Each identifier is associated with the memory address of the object to which it referring to. An identifier can be associated with one type of object initially and later it can be reassigned to another object that is of the same or different type.

```
x = [10, 20]
y = x                    # y now refers to the same list object as x
print(x is y)            # Output: True (since x and y are aliases)
```

Let us say, we decide to add another number to the list then the scenario is:

```
y.append(30)             # Modifying y also affects x

print(x)   # Output: [10, 20, 30]
print(y)   # Output: [10, 20, 30]
```

In the example illustrated above, x and y are aliases because they point to the same list. Modifying any one of the the list will affect the other. These characteristics should be borne in mind when working with mutable objects like lists and dictionaries. Aliasing can lead to unexpected side effects if not handled carefully. To avoid any unintended effect, one can create a copy of the object instead of creating an alias.

```
x = [10, 20]

y = x.copy()             # y is now a copy of x

print(x is y)            # Output: False (x and y are different objects)

y.append(30)
print(x)   # Output: [10, 20]
print(y)   # Output: [10, 20, 30]
```

1.6 EXPRESSIONS IN PYTHON

Like in C or C++, a combination of operands and operators is called an expression. The expression produces some value or result after being interpreted by the Python interpreter. It combines operators, variables, literals, and function calls to produce a value. The programming language uses distinct expressions for various requirements. Decision-making requires Boolean values from logical expressions like and, or. As you know, relational expressions compare values and return Booleans in Python, while arithmetic expressions compute numbers.

Complex expressions may involve numerous operators and types. The sequence of operation is determined by the Operator precedence inthese expressions. A defined operator precedence hierarchy in Python ensures proper expression evaluation. Programmers have additional control by changing the evaluation order with parentheses. The program efficiency depends on using expression types and operator precedence rules.

Table below illustrates various operators from the highest to the lowest precedence along with its description and the associativity of the operator.

Precedence	Operator	Description	Operator Associativity
1	()	Parentheses	Left to right
2	x[index], x[index:index]	Subscription, slicing	Left to right

3	await x	Await expression	N/A
4	**	Exponentiation	Right to left
5	+x, -x, ~x	Positive, negative, bitwise NOT	Right to left
6	*, @, /, //, %	Multiplication, matrix, division, floor division, remainder	Left to right
7	+, –	Addition and subtraction	Left to right
8	<<, >>	Shifts	Left to right
9	&	Bitwise AND	Left to right
10	^	Bitwise XOR	Left to right
11	|	Bitwise OR	Left to right
12	in, not in, is, is not, <, <=, >, >=, !=, ==	Comparisons, membership tests, identity tests	Left to Right
13	not x	Boolean NOT	Right to left
14	and	Boolean AND	Left to right
15	or	Boolean OR	Left to right
16	if-else	Conditional expression	Right to left
17	lambda	Lambda expression	N/A
18	:=	Assignment Operator	Right to Left

```
# Illustrating Operator precedence

value = 2 + 4 * 6

print (value)
```

➔ Output of print statement shall be 26. This is because multiplication operation has higher precedence and hence 4 * 6 gets computed first as 24. Later 24 is added to 2 to produce 26 as output.

```
# Illustrating Operator Associativity

value = 2 * 4 / 6

print (value)
```

It can be seen here that both * and / have same precedence but the associativity tells us that the expression shall be evaluated from left to right. Hence Multiplication is performed first and later division.

➔ Output of print statement shall be 0.48.

Constant Expressions

An expression in Python that contains only constant values is known as a constant expression. In a **constant expression** , the operator(s) is a constant. A constant is a value that cannot be changed after its initialization.

Example:

```
val = 2 + 4      # both 2 and 4 are constants although val is a variable
```

➔ Output is 6

1.6.1 Integral Expressions

An integral expression in Python is used for computations only after necessary type conversion (integer to float, a string to integer, etc.).

Example:

```
x = 2          #x is assigned with 2 which is an integer number

y = 4.0        #y is assigned with 4.0 which is a floating point number
```

In order to carry out addition of these two variables, either we convert int to float and add two float numbers or we convert float to int and add two int numbers.

```
val = x + int(y)          # y type converted to int before addition operation
```

➔ Output is 6

or

```
val = float(x) + y        # x type converted to float before addition operation
```

➔ Output is 6.0

1.6.2 Relational Expressions

These are used in expression evaluation to compare two operands and return a Boolean value that is either True or False based on the evaluation of the expression. These operators find use in decision flow and conditional logic in programming used in if statements and loops.

Common relational operators in Python are listed below:

Less than	<	(e.g., x < y)
Less than or equal to	<=	(e.g., x <= y)
Greater than	>	(e.g., x > y)
Greater than or equal to	>=	(e.g., x >= y)
Equal to	==	(e.g., x == y)
Not equal to	!=	(e.g., x != y)

 # Illustrating relational expressions

```
x = 2
y = 4

print(x < y)      # Output: True
print(x <= 5)     # Output: True
print(x >= y)     # Output: False
print(y == 4)     # Output: True
print(x != y)     # Output: True
```

1.6.3 Logical Expressions

These performs the logical computation and the value of the overall expression results in either True or False i.e. boolean in nature. These operators are crucial for controlling program flow and making decisions based on multiple conditions in the expression. Python has three main logical operators namely and, or, and not.

```
x = 20
```

```
print(x > 2 and x < 25)

# This returns True because 20 is greater than 2 AND 20 is less than 25.
```

or Operator

```
x = 100

print(x > 50 or x < 20)

# This returns True because one of the conditions are true.

#100 is greater than 50, but 100 is not less than 20.
```

not Operator

```
x = 10

print(not(x > 2 and x < 25))

# This returns False because not operator reverses the result of and.
```

1.7 CONTROL FLOW

Program execution happens sequentially in Python wherein the Python interpreter reads a code written by you line by line from top to bottom with each statement interpreted from left to right and. The interpreter executes operations and functions in the order that it reads which is what the control flow signifies**.** Not every program shall follow this pattern since the program shall also encounter conditional statement and based on this outcome, the control gets transferred to the desired statement. Without control flow expressions, a program is simply a list of statements that are sequentially executed. Python uses if, elif (short for else if), and else keywords to create control flow structures in your program. Imagine all these keywords are traffic controllers telling the control flow where to go. Thus a program's control flow decides the order in which the program code executes. Python has three types of control structures which are listed as under:

Sequential: Default mode if no other branch logic implemented

Selection: Used for decisions in conditional statements and branching

Repetition: Iteration with the looping where asection of code gets executed the said number of times.

1.7.1 Sequential Statements

These are the statements where execution happens in a sequence. The problem with sequential statements is that if the logic has broken in any of the lines, the complete source code execution will fail.

```
# The following piece of code represents a Sequential statement

a=20
b=10
c=a-b
print("Subtraction of the two values is : ",c)
```

➔ Output
Subtraction of the two values is: 10

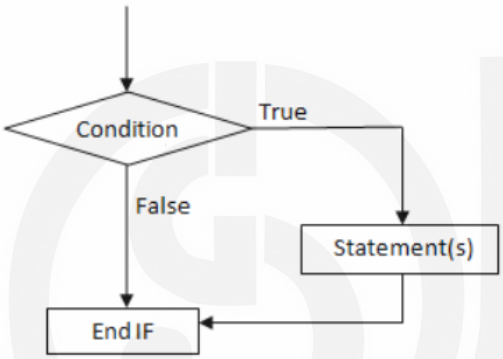
1.7.2 Decision Control Statements

When a program code statement or set of statements is executed, a condition evaluates to True or False is referred to as Decision Control Structure. Whether a section of code is executed or not is decided by checking a condition. Some decision control statements are as listed as under:

- if
- if-else
- nested if
- if-elif-else

if Statement

The if statement a basic decision control statement used in decision making. It helps to run a particular code only when a certain condition is true. The same is illustrated by a figure below.




```
# Program to increment a number if it is positive

n = 100          # Initialize the value of n
if (n>50):       # test the value of n
n = n+1          # Increment the value of n if n is > 50
print(n)         # print the value of
```

➔ Output

101

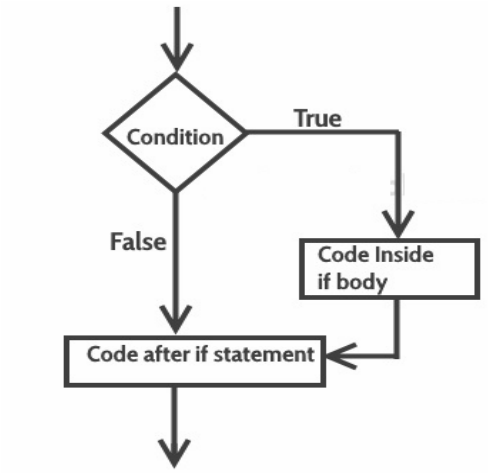
if-else

If-else statement is a control structure that ensures a program to execute different blocks of code based on the evaluation of a condition that evaluates to true or false. It checks a condition, and if it's true, the code block within the if statement is executed and if the condition is false, the code block after the else statement is executed. One should remember that Python uses indentation in both the blocks to define the scope of the code. Other programming languages often use curly brackets {} for this purpose. The same is illustrated below.

```
if(condition):
        #Executes this block if the condition is true
```

```
else:
        #Executes this block if the condition is false
```

The flow of decision control is illustrated in the figure below.




```
# Program to increment a number if it is positive

marks = 71
if marks > 60:
  print("First Division")
else:
  print("Second Division ")
n is odd
```

➔ Output

```
First Division
```

Nested if or if .. elif .. else Statement

Nested **if** statements in Python allow you to have an if statement within another if statement. This is useful for scenarios where you need to evaluate multiple conditions during the execution of the program. These are an efficient way for coding complex / nested decision-making logic in your programs. Remember that one must ensure proper indentation and keep the code readable, understandable and maintainable. The same is illustrated here.

```
if (condition1):
        #Program Statements
elif (condition2):
        #Program Statements
. elif (condition3):
        #Program Statements
else:
        #Program Statements
```

```
# Compare the two numbers using nested if (elif) statements

x = 100                 # value 100 assigned to x
y = 200                 # value 200 assigned to y
if x > y:
   print("x is greater than y")
elif:
```

```
if x < y:
    print("y is greater than x")
else:
    print("x and y are equal")
```

➔ Output

y is greater than x

1.8 LOOPING / REPETITION

Understanding loops is essential for any python programmer as these provide a powerful tool to automate code logic and process data. Python being popular programming language, offers several loop structures to iterate over sequences and execute repetitive tasks efficiently. These are also referred to as repetition statement used to repeat a block of code statements. Python has two loops structures which are:

for loop

while loop

1.8.1 for loop

It is used to iterate over a sequence that can be a list, tuple, dictionary, or a set. It executes the block of code for each item in the sequence until the last item in sequence. This loop starts with the for keyword, followed by a variable that represents the current item in the sequence. The 'in' keyword links the variable to the sequence you want to iterate over. A colon ': is used at the end of the loop header, and the indented block of code is executed once for each item in the sequence.

for item *in* sequence:

Code block to be executed for each item

Let us understand this with a sample program code illustrated below.

```
# Program to find the sum of all elements in a given list

elements = (1,2,3,4,5)
total = 0
for item in elements:
        total = total + item
print ("Total of all elements in the given list =", total)
```

➔ Output

Total of all elements in the given list = 15

Let us get a brief insight into the following python code snippet, it illustrates how to use a for loop to iterate over a list and access its elements, while the second example shows how to iterate over a range of numbers.

```
# Program to iterate on a given list and print elements of a given list

givenlist = [1, 2, 3]
for i in range(len(givenlist)):
    print(givenlist [i], end = " \n")
```

➔ Output

```
1
2
3
```

Program to iterate on a range of numbers

```
for elements in range(0,4):
   print(elements, end = " \n")
```

➔ Output

```
0
1
2
3
```

Note that the end parameter in the print() function specifies the character or string that should be printed at the end of the output. The default value is "\n" (newline character), ensuring each value to be printed on a new line.

1.8.2 while and do-while loop

The while loop in Python executes a body or block of code repeatedly until the condition is true. It is useful when the number of iterations is not known to the programmer in advance. The condition is evaluated before each iteration over the block of code. If it is true, the code block inside the loop is executed. This process continues until the condition becomes false. One should ensure to add certain termination condition like break.

```
while   condition:
                # Body / Program Code to be executed repeatedly
```

Python code to illustrate while loop where the code iterates from 0 to 4 and prints each value using . Finally, it prints "End of list" to signify the end of the program after the loop is completed.

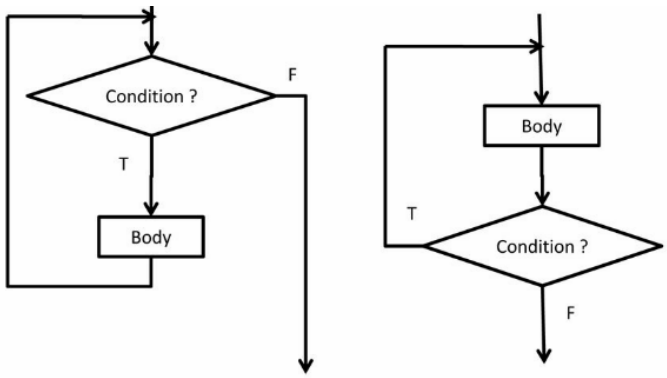
```
number = 5
i = 0
while i < number:
    print(i, end = " ")
    i = i + 1
print("End of List")
```

➔ Output

```
0 1 2 3 4 End of List
```

It is worth mentioning that Python does not have a built-in do-while loop like other programming languages. However, it is possible to exhibit the similar effect of a do-while loop using a while loop along with a break statement.

The only difference worth remembering is that in a do-while loop, the code block is executed at least once, and then the condition is checked. Thus, if the condition is true, the loop continues else it terminates. The same is illustrated in the figure below.




```
# Program to illustrate do-while loop and its termination

number = 1
while True:
        print(number)
        number = number + 1
        if number > 10:
                break
```

➔ Output

```
1
2
3
4
5
6
7
8
9
10
```

1.9 FUNCTIONS

A function is a block of reusable code that performs a specific task. This task is generally one which is used multiple times to develop an application in Python. Suppose we need to create a program to compute area of a given land or the volume of a box. The idea is to organize code, make it more readable, and avoid repetition of the same code statements. Thus, it helps to divide a complex problem into smaller ones and makes our program code easy to understand and reuse. Let's take a quick glimpse of how a function works.

Creating a Function

```
        def welcomemessage ():

                print("welcome to the world of Python")
```

The keyword "def" is used to create a function. welcomemessage () is the name of the function. The print statement forms the function body. Now in order to use or call this function, the following program gives you the insight.

```
# program to define a Function and call it

def welcomemessage ():

                print("welcome to the world of Python")
```

```
welcomemessage()
```

➔ Output

```
welcome to the world of Python
```

Lets take a more relevant program that computes area. It takes two parameters. The length and the breath. Later, a function calls it using the necessary arguments.

```
# Program to define a Function for computing Area and calling it

def area(length, breath): #define the Function area
   return length * breath # Body of function Area
areaofroom = area(10, 5)
print(areaofroom)
print(area(10, 10))                    #Calling function to compute area
```

➔ Output

```
50
100
```

There are four major types of functions in Python. These along with the use cases are listed below.

1.10 BUILT-IN FUNCTIONS

Functions that are available in Python, such as print(), len(), sum(), int(), etc.

User-defined functions

Functions created by the programmer to perform specific tasks. These are defined using the def keyword, followed by the function name, parentheses (), and a colon : as illustrated for the function to compute the area().

Lambda functions

A lambda function is a special type of function without the function name. It is a one-line functions defined using the lambda keyword. These are generally used for simple operations. Functions like welcomemessage () discussed above qualify for lambda functions.

Recursive functions

A recursive function in Python are the functions that calls itself within its definition. Each recursive function must have two key components namely a base condition and a recursive step like in any other programming language..

Base Case

This is the condition that stops the recursion and prevents infinite loops. Without this condition, the function can call itself indefinitely, leading to a stack overflow.

Recursive Step

This is where the function calls itself like in calculating Factorial of a number.

Python supports both Method and Function. As we have seen, Methods are dependent on objects and are called by their name. A method definition always includes ‘self’ as its first parameter and is implicitly passed to an object on which it is invoked. It may

or may not return any data. Although, Function is block of code that is also called by its name but function can have different parameters based on the application desired or may not have any at all. If any data parameters) are passed, they are passed explicitly. It may or may not return any data. Finally, Functions don't deal with Class and objects.

Python program to illustrate Recursive Function

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)

print(factorial(4))
```

➔ Output

24

Base Case and Recursive Case of Factorial(n)

Base Case: This is the condition under which the recursion should stop. It is crucial to prevent infinite loops and to ensure that each recursive call reduces the problem in some manner. In the factorial() code illustrated above, the base case is n == 1 since when n ==0, it returns and comes out of the loop.

Recursive Case: This is the computing part of the function calls to itself. It must eventually lead to the base case. In the factorial example, the recursive case is return n * factorial(n-1).

Simple Input and Output in Python

In Python, taking input from the console and displaying the output are fundamental for interacting with users and displaying information while executing any code. The input() function ensures reading data from the console, while the print() function enables displaying output.

Input

The input() function prompts the user for a string of text once you execute a program. It takes argument that is displayed as a prompt to the user. The function then waits for the user to enter text and press Enter. The entered text is returned as a string.

Remember that every console input in Python obtained via the input() function is always taken as a string. If we have to use the input as a different data type, such as an integer or float, it is necessary to explicitly convert the input using type casting functions like int() or float(). If the input is intended to be used as a string, no explicit conversion is required. The same is illustrated by a Python program below.

Python program to illustrate use of input() function to take user input

```
num = input('Enter a number: ')
print('You Entered:', num)
print('Data type of num:', type(num))
```

➔ Output

Enter a number: 125

You Entered: 125

Data type of num: <class 'str'>

1.11 TAKING MULTIPLE INPUTS FROM CONSOLE

When we require multiple inputs from the user in a single line, splitting the values entered by the user into separate variables for each value is necessary and the same is achieved by using the split() method of Python. The same is illustrated by a Python code below.

```
# Python program to illustrate taking multiple inputs at a time
x, y = input("Enter two values: ").split()
print("Number of cars: ", x)
print("Number of bikes: ", y)

# Illustrating Concatenation of strings taken as Console Inputs
y, z = input("Enter three values: ").split()
total = y + z
print("Total number of vehicles: ", total)
print("Number of cars is: ", y)
print("Number of bikes is: ", z)
```

➔ Output

```
Enter two values: 5 6
Number of cars:  5
Number of bikes:  6
Enter three values: 5 6
Total number of vehicles:  56          # The two strings 5 and 6 are
concatenated
Number of cars is:  5
Number of bikes is:  6
```

```
# Typecasting Console Inputs to obtain the Sum
y, z = input("Enter two values: ").split()
total = int(y) + int(z)
print("Total number of vehicles: ", total)
print("Number of cars is: ", y)
print("Number of bikes is: ", z)
```

➔ Output

```
Enter two values: 5 6
Number of cars:  5
Number of bikes:  6
Enter three values: 5 6
Total number of vehicles:  11
Number of cars is:  5
Number of bikes is:  6
```

1.12 EXCEPTION HANDLING

Python exception handling provides us with a backup plan for our code in a situation when something unexpected happens during the execution. Hence it can be said to be an exception that occurs during program execution. For example, dividing number by 0 raises exception called a ZeroDivisionError. Whenever these types of runtime errors occur, Python creates an exception object. For serious programmers, exception handling allows developing a robust and reliable code. By planning and handling these types of errors and unexpected situations, we ensure that our code continues to function correctly during unexpected events. With try, except, and finally blocks, Python provides powerful tools for managing errors and maintaining the integrity of our code to create maintainable software applications.

Lets us see an exception situation depicted in a code snippet below. We don't provide for any specific exception handling and hence the output of this prints a traceback to that error along with some details about why the error has occurred as shown below.

```
# Need of Exception Handling

divide_by_zero = 7 / 0

print(divide_by_zero)
```

➔ Output

```
ERROR!

Traceback (most recent call last):

  File "<main.py>", line 1, in <module>

ZeroDivisionError: division by zero
```

try…except…finally in Python

In Python, the 'try...except...finally' statement is an extended version of the **'try...except'** statement.

Try Block: This is where we write the code that might cause an error or exception condition.

Except Block: This is where we write the corrective backup plan in case an error happens in the try block.

Finally Block

This block always runs irrespective of the fact that an exception occurrsr not. It's like putting thigs right after handling the exception. It guarantees predictable code behaviour that creates maintainable code. Unlike the try-except block, which handles exceptions, the try-finally block focuses on corrective cleanup operations that must occur, ensuring resources are properly released and the intended tasks are completes.

Now lets take up the same code that tries to divide a number by zero and implement the same with try..except..finally.

```
# Program with no exception and role of Finally

try:

    number = 7 / 1

except ZeroDivisionError:
```

```
    print("Can't divide a number by zero")
finally:
    print('This part of code is always executed')
```

➔ Output

This part of code is always executed

```
# Program with exception and the role of Finally
try:
    number = 7 / 0
except ZeroDivisionError:
    print("Can't divide a number by zero")
finally:
    print('This part of code is always executed')
```

➔ Output

Can't divide a number by zero

This part of code is always executed

1.13 Built-in Exceptions in Python

For the sake of insight, the table below illustrates the built in exceptions that are available in Python (in albhabetic order).

ArithmeticError	error due to numeric calculations
AssertionError	when an assert statement fails
AttributeError	attribute reference or assignment fails
Exception	Base class for all the exceptions
EOFError	when input() method encounters an "end of file" condition (EOF)
FloatingPointError	Due to a floating point calculation failure
GeneratorExit	when a generator is closed
ImportError	when an imported module is not found
IndentationError	indentation incorrect
IndexError	index of a sequence does not exist
KeyError	key not found in a dictionary
KeyboardInterrupt	Due to pressing Ctrl+c, Ctrl+z or Delete
LookupError	when errors raised is not found

MemoryError	Program memory runs out
NameError	variable does not exist
NotImplementedError	abstract method requires an inherited class to override any method
OSError	system related operation causes an error
OverflowError	result of a numeric calculation is too large
ReferenceError	reference to weak object does not exist
RuntimeError	error that do not belong to any specific exceptions
StopIteration	next() method of an iterator has no values left
SyntaxError	syntax error occurs
TabError	Due to indentation consists of tabs or spaces
SystemError	Due to a system error
SystemExit	Calling the sys.exit() function
TypeError	Combining two different types
UnboundLocalError	Referencing local variable before assignment
UnicodeError	unicode problem found
UnicodeEncodeError	unicode encoding problem found
UnicodeDecodeError	unicode decoding problem found
UnicodeTranslateError	unicode translation problem found
ValueError	wrong value found in a specified data type
ZeroDivisionError	second operator in a division is found to be zero

1.14 ITERATORS AND GENERATORS

Iterators and generators in Python provides the programmers to iterate over the data one item one at a time without having to load all of the data together into memory at once. This is particularly helpful when working with large datasets for efficient memory management.

Iterators in Python

An iterator is an object that allows the programmers to traverse / iterate or loop through a collection of items like a list one item at a time.

The Iterator Protocol

Iterators follow certain rules that constitutes the iterator protocol. This consists of two main methods listed below:

__iter__():

This method initializes the iterator and returns it.

__next__():

This method returns the next item in the collection. If there are no more items, it issues a StopIteration error to signal that the iteration is complete.

```
class IteratorExample:
    def __init__(self, item):
        self.item = item                    # To iterate over the given list
        self.index = 0                      # Initializing the starting index in the list
    def __iter__(self):
        return self                         # Returns the iterator object itself
    def __next__(self):
        if self.index < len(self.item):     # To check if there are more items in the list
            # Get the current item
            value = self.item[self.index]
            self.index += 1
            return value                    # Return the current item
        else:
            raise StopIteration
# Usage of IteratorExample
prime_list = [1, 2, 3, 5, 7, 11]  # A simple list
iterator = IteratorExample(prime_list)      # Create an iterator for the list
# Loop through the items using the Iterator
for number in iterator:
    print(number)  # Print each item
```

➔ Output

```
1
2
3
5
7
11
```

1.15 GENERATORS IN PYTHON

A generator is a simpler way to create an iterator using a function that outputs certain values. When a generator function is called, it returns a generator object, which behaves like an iterator. Each time the next() function is invoked on the generator object, the function body executes until a yield statement is encountered. The value

produced by yield is returned, and the function's state is saved. Subsequent calls to next() resumes the execution of the function from the last saved state, until the function exits or raises a StopIteration exception.

```
def generatorExample(item):
    for number in item:                          # Loop through each item in
the data
    yield number                                 # Yield (return) the current
item and pause

# Usage of my_generator
prime_list = [1, 2, 3, 5, 7, 11]                 # A simple list
generator = my_generator(prime_list)             # Create a generator

for number in generator:                 # Loop through the items produced by the
generator
   print(number)                                 # Print each item
```

➔ Output

```
1
2
3
5
7
11
```

So, the above code shows how yield() pauses the function execution and returns a value. It saves the variable values and the function execution point and resumes from this point when the generator is called.

1.16 ADDITIONAL PYTHON CONVENIENCES

Python's convenience lies in its vast number of libraries for Data Science (statistical), Machine learning, Deep learning and NLP models illustrated in tables below. Python's widespread use is noty only because of its simplicity, it is also because of its various modules and libraries, each tailored to make a developer's coding efficient and scalable code.

Python Libraries for Data Science

Library Name	Use
NumPy	scientific computation
Pandas	data analysis, data manipulation, and data cleaning
Matplotlib	interactive, and animated Python visualizations
Seaborn	data visualization framework

Python Machine Learning Libraries

Library Name	Use
LightGBM	open-source gradient boosting library that employs tree-based algorithms
XGBoost	implementation of machine learning algorithms within

	the gradient boosting framework
Statsmodels	estimate various statistical models, conduct statistical tests, and do statistical data exploration
Optuna	automate hyperparameter searches

Python Deep Learning Libraries

Library Name	Use
TensorFlow	end-to-end open-source platform for machine learning
PyTorch	optimized tensor library for deep learning using GPUs and CPUs, and is considered to be an alternative to TensorFlow
Keras	Most popular and supports nearly all models of a neural network, including convolutional, embedding, pooling, recurrent networks

Python Libraries for Natural Language Processing

Library Name	Use
NLTK	parse human language data - called "a wonderful tool for teaching, and working in, computational linguistics using Python"
spaCy	natural language processing library in Python - excels at large-scale information extraction tasks
Gensim	topic modeling, document indexing, and similarity retrieval with large data set
Hugging Face Transformers	Pre-trained models for text, audio, images and multimodal

1.17 SCOPES AND NAMESPACES

A namespace is a collection of names where we have a mapping of every name we have defined to corresponding objects in the code. An object can be a variable or a method. Name represents a unique identifier and Space refers to the scope of the name. Python itself maintains a namespace in the form of a Python dictionary. At the highest level of hierarchy, we have built-in name space like print(). This is followed by Global name space. When a user creates a module, a global namespace is created and further, when we write local functions, it creates the local namespace. Thus,a local namespace is created when a function is called and this has all the names defined in it. Hence, the built-in namespace encompasses the global namespace and the global namespace encompasses the local namespace. The same is illustrated by a Python program below.

```
# Program to illustrate usage of Global, Local and Nested Namespaces

global_var_val = 2                  # global_var represents global namespace
def outermost_function():
        outer_var_val = 4                   #  This variable is in the local namespace
    def inner_function():
                inner_var_val = 6           #  This is in the nested local namespace
        print(inner_var_val)
    print(outer_var_val)
    inner_function()
print(global_var_val)               # Prints the value of the global variable
```

```
outermost_function()    # Calls the outer function that includes  local and nested local variables and prints it
```

➔ Output

```
2
4
6
```

1.17.1 Scope of Objects in Python

One cannot access any particular object from anywhere in the code, the same is defined by the scope of the object. Scope refers to the part of the program from where a particular Python object can be referenced. This translates to the portion of a program from where a namespace can be accessed directly without any prefix. At any point of interpreting a code, there are at least three nested scopes mentioned below.

Scope of the function in reference which contains local names searched in local namespace

Scope of the module which has global names searched in the global namespace

Outermost scope which has built-in names and namespaces

It is worth mentioning that the lifetime of a namespace depends upon the scope of objects being referenced. If the scope of an object ends, the lifetime of that namespace also comes to an end.

1.17.2 Modules and the Import Statement

Most of us have used "#include" while writing the C or C++ codes. This file includes numerous methods / functions that makes coding easier. Similarly, a Module is a file that contains methods / functions code to perform various tasks. These modules could be built in facility of python or one can also create own Module that stores all the methods / functions that you frequently use during coding. A module may contain variables, functions, classes etc. As the LoC (Lines of Code) increases, program size grows bigger. Thus, instead of putting everything in a single program file, we use modules to call the required methods / functions as per the functionality required. This makes our code understandable and maintainable.

User Defined Modules

Let us create a module that stores function for multiplying two numbers and save the file as multiplyexample.py. Now, one can import the definitions that exist inside a module to another module or the interactive interpreter in Python where you execute the code. This is done by the use of 'import' keyword.

```
# Python program to create a sample Module

def multiply(a, b):
   result = a * b
   return result
```

Now, to use this module, we can go to our Python prompt and type:

```
Import multiplyexample
multiplyexample.multiply(5,4)   # Calling multiply function in multiplyexample
```

➔ Output

```
20
```

Similarly, the Python library built-in modules or any specific function in these modules can be used by the "import" statement.

```
# Illustrating usage of importing Math Module of Python
```

```
import math
cube = pow(2, 3)                # Finding Cube of a number
print(cube)
```

➔ Output
8

To make efficient usage of the resources, one can import the specific functions or variables that is required in our code using the from keyword instead of importing the entire module. The same is illustrated below.

```
# Illustrating usage of importing specific function from Math Module of Python

from math import pow
cube = pow(2, 3)                # Finding Cube of a number
print(cube)
```

➔ Output
8.0

One can shorten the name of the module being imported although it shall promote cryptic programming. Nevertheless, this is achieved by the use of "alias" as illustrated here.

```
import math as aa
cube = aa.pow(2, 3)
```

1.18 SUMMARY

In recent years, Python has become a language of choice since it has simple syntax, easier code readability, user friendly and can be used with wide variety of applications. It has extensive library, large active user community, cross-platform compatibility and beginner-friendly design philosophy. To understand its acceptability, for web and software development, Python is behind many websites and applications. For understand the hidden data patterns, Data analysts use Python's libraries to process and visualize huge datasets. Python is a dynamic, interpreted (can also be referred to as byte code-compiled) language. This means that prototyping can be very quick while testing the code.

Python is an object-oriented language and classes form the basis for all its data types. Classes are a means of bringing together data and functionality together. In Python, an identifier is a name given to a programming element, such as a variable, function, class, or module. It allows programmers to refer to these elements within their code. It is worth mentioning that Python does not have a built-in do-while loop like other programming languages. However, it is possible to exhibit the similar effect of a do-while loop using a while loop along with a break statement. The keyword "def" is used to create a function. A function is a block of reusable code that performs a specific task.

Every console input in Python obtained via the input() function is always taken as a string. If we have to use the input as a different data type, such as an integer or float, it is necessary to explicitly convert the input using type casting functions like int() or float(). If the input is intended to be used as a string, no explicit conversion is required. Python exception handling provides us with a backup plan for our code in a situation when something unexpected happens during the execution. Iterators and generators in Python provides the programmers to iterate over the data one item one at a time without having to load all of the data together into memory at once. This is particularly helpful when working with large datasets for efficient

memory management. Python's convenience lies in its vast number of libraries for Data Science (statistical), Machine learning, Deep learning and NLP models.

The built-in namespace encompasses the global namespace and the global namespace encompasses the local namespace. The modules could be built in facility of python or one can also create own Module that stores all the methods / functions that you frequently use during coding. A module may contain variables, functions, classes etc. As the LoC (Lines of Code) increases, program size grows bigger. Thus, instead of putting everything in a single program file, we use modules to call the required methods / functions as per the functionality required. This makes our code understandable and maintainable.

Check your Progress

1. What are the benefits of using Python?
2. What operators does python support?
3. What are the common built-in data types in Python?
4. Where and how is a Python function defined?
5. What is a Try block in Python?
6. What is recursion in Python?
7. What is meant by module in python?
8. Write few names of Python built-in modules?
9. What are the common built-in data types in Python?
10. What is Scope in Python?
11. How good is Python for data analysis?
12. What are Python namespaces? Why are they used?
13. Is Indentation Required in Python?
14. What is a namespace in Python?
15. What is the difference between .py and .pyc files?

1.19 Further Readings

1. Python Essentials You Always Wanted to Know: Beginner's Guide to Python Programming, Data Structures, Data Analytics with Hands-On Coding Exercises

by Shawn Peters and Vibrant Publishers

2. Python Programming : Beginners Guide Paperback

by Sundarrajan M (Author), Mani Deepak Choudhry (Author), Jeevanandham S (Author), Akshya Jothi (Author)

3. Python Crash Course: A Hands-On based Introduction to Programming by Eric Matthes; No Starch Press (Publisher); ISBN-10: 1718502702; ISBN-13: 978-1718502703

4. Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter by Wes McKinney; O'Reilly (Publisher); ISBN-10: 9355421907; ISBN-13: 978-9355421906

Reference Websites

https://pythoninstitute.org/python-essentials-1

https://pythoninstitute.org/python-essentials-2

ignou
THE PEOPLE'S
UNIVERSITY

UNIT 2 OBJECT ORIENTED PROGRAMMING

Structure

2.0 Introduction
2.1 Objectives
2.2 Principles and Patterns
2.3 Python Objects
2.4 Abstract Class
2.5 Types of Inheritance
2.6 Namespace and Object-Orientation
2.7 Python Lists
2.8 Copy An Object in Python Using '='
2.9 Summary
2.10 Further Readings

2.0 INTRODUCTION

The object oriented programming (OOP) paradigm is concerned with real world objects and its roots lie in modeling and later writing code around real world objects. These objects are logically linked together to form an internal hierarchy. In programming realization, these real world objects are represented by classes and the hierarchy defines the relationship between these classes. A programmer chooses the data representation that best fits the requirements. Thus a program is structured around the data rather than operations.

In the very simplest terms, objects can be defined as reusable software components that model items in the real world. These are meaningful software units such as student objects modeling the general characteristics of a student, date objects, time objects, invoice objects, etc. These are reusable and more understandable, better organized and modular and easier to maintain than procedural programming.

The main idea behind Procedural Programming is to decompose a task into further subtasks. This process of decomposition is repeated until the subtasks can be appropriately coded. A solution to a problem is thus a conglomerate of functions and subprograms, communicating through arguments and variables that may have a global or local scope. Thus a program is designed and structured based on control flow rather than data flow giving more weightage to program than data.

2.1 OBJECTIVES

The purpose of this unit is to introduce Object Oriented Programming as Python is an Object Oriented Programming language.

After studying this unit, you should be able to understand :

- OOPs concepts
- Python as OOP language

2.2 PRINCIPLES AND PATTERNS

Principles and patterns are useful in defining fundamental design concepts in object-oriented programming (OOP). These act as basis for the design and implementation of robust and maintainable programs. Lets first understand the necessary principles of desiging classes in Python.

- **Designing Classes & Class Hierarchies**

  Defining and defining any class or class hierarchies requires one to follow certain principles in order to produce a program code that is not only efficient but also minimizes overlap of functionalities among classes and redundancies. Certain important principles that one should follow are listed here.

- **Removing Duplicity of Data and Function**

  Functions defined in one class should not be duplicated in other classes. If one has defined a function named "area" for calculating area of a room, then one need not define the same function again as it can be available through inheritance discussed in the sections below.

- **Objects as Real World Entities**

  Every real world entity should be represented by a distinct class in the program. This will ensure that the design is efficient with well connected components. Ensure that a class does not pack too much functionality. This makes understandability and maintainability difficult since it demands more cognition. Frequently needed computations qualify for being a function in a class.

- **Access Mechanisms** – Public, Protected & Private

Class Variables are the variables that are shared across all instances of a class. It is defined at the class level, outside any methods. Instance Variables are unique to each instance (object) of a class that are defined within the __init__ method or other instance methods.

- **Public**

  Public members namely attributes / variables and methods/functions are accessible from anywhere, both within and outside the class. By default, all attributes / variables and methods/functions in a Python class are public.

- **Protected**

  Protected members are intended for use within the class and its subclasses. They are denoted by a single underscore prefix _ like "_protected_method()".

- **Private**

  Private members are intended for use only within the class. They are denoted by a double underscore prefix __ like "__private_method()".

Classes are created by keyword class. Attributes are the variables that belong to a class. These can be accessed using the dot (.) operator.

2.3 PYTHON OBJECTS

An Object is an instance of a Class in Python as is the case with other object oriented programming languages. It represents a specific implementation of the class and holds its own data. An object consists of:

Identity provides a unique name to an object and enables one object to interact with other objects.

State represented by the attributes and exposes the properties of an object.

Behavior represented by the methods of an object and provides the response of an object to other objects.

Having deliberated upon the principles of designing classes in Python, let us explore about the core principles of OOP in Python.

Encapsulation refers to bringing together data / attributes and methods / functions defined within a class. This principle restricts access to certain components to control interactions by the way these are defined within a class. Hence three types of encapsulations practiced in Python are as under**:**

- Public Members: These are accessible from anywhere in the program.
- Protected Members: These are accessible only within the class and its subclasses.
- Private Members: These are only accessible within the class.

Bundling data (attributes) and methods that operate on that data within a class, controlling access and preventing unintended modification.

Inheritance is fundamental to object oriented programming wherein a new class, known as a child or derived class, acquires attributes and methods from an existing class. This existing class is known as the base class or parent class. This very property of OOP promotes code reuse and facilitates creation of hierarchical relationships between parent and derived classes. The derived class can inherit and use the properties and behaviors of the base class and also extend or modify them. But, this depends on the type of encapsulation.

The newly created class is known as the subclass also referred to as child or derived class. The existing class from which the child class inherits is known as the superclass also referred to as parent or base class. Python supports both single and multiple inheritances. The same is illustrated by a pseudo-code here.

```
# define a superclass
class super_class:
   # attributes and method definition

# Sub_Class Inheriting the Super_Class
class sub_class(super_class):
   # attributes and method of super_class are inherited
   # attributes and method of sub_classs also available
```

Polymorphism is a property that allows methods to carry same name as in super class but have different functionality based on the object's context. This is achieved

through method overriding or overloading. Polymorphism facilitates dynamic method usage, where the specific method to be executed is determined at runtime based on the object's actual type. This actually translates into a fact that one can call a method on an object without knowing its exact type in advance. Polymorphism facilitates that the system will automatically choose the appropriate implementation based on the object's class. The two basic types of Polymorphism are discussed in this section.

Compile-time polymorphism also referred to as static polymorphism or method overloading, is a behavior where the specific method to be executed is determined at compile time. Although Python does not support traditional method overloading in the same way that languages like C++ or Java, but similar behaviour also exists through techniques like default arguments or variable-length argument lists.

In languages that provide built-in compile-time polymorphism, multiple methods with the same name can be defined by the programmer but varying parameter lists that could be either the number of parameters or the type of parameters. It is the compiler that selects the appropriate method to call based on the arguments provided in the method / function call.

Python, being a dynamically typed language, resolves method calls at runtime also termed as method overloading. If we have multiple methods with same name in a class, the last definition of a function / method shall override any of its previous definitions. The same is illustrated by a python program below.

```
# Python program to illustrate Compile-Time Polymorphism using method overloading

# First method named sum()
# It accepts two argument and prints the sum of two numbers
# product

def sum(a, b):
    total = a * b
    print(total)
# Second method named sum()
# It accepts three argument and prints the sum of three numbers

def sum(a, b, c):
    total = a * b*c
    print(total)

sum(10, 20)
sum(10, 20, 30)          #second definition of sum() with three parameters
```

➔ Output

```
Hangup (SIGHUP)
Traceback (most recent call last):
  File "/home/guest/sandbox/Solution.py", line 16, in <module>
    sum(10, 20)
    ~~~^^^^^^^^
TypeError: sum() missing 1 required positional argument: 'c'
```

The error message appears because the last definition of the method sum() expects three arguments. When it encounters "sum(10, 20)", the compiler finds that the third argument is missing and hence the error message.

Run-Time Polymorphism is a type of polymorphism is determined during the execution of the program. It occurs when a subclass contains a implementation for any / all method(s) already defined in its parent class, commonly known as method overriding. Some commonly practiced polymorphism in Python are the following:

- Polymorphism in Built-in Functions
- Polymorphism in Functions
- Polymorphism in Operators
- Polymorphism in Object-Oriented Programming (OOP)

 # Python program to illustrate Polymorphism in Functions

```
print(len("Good Morning"))          # String length
print(len([10, 20, 30, 40]))        # List length
```

➔ Output

```
12
4
```

 # Python program to illustrate Polymorphism in Functions

```
def add(a, b):
    return a + b

print(add(10, 20))                  # Integer addition
print(add("Good " , "Morning"))     # String concatenation
print(add([10, 20], [30, 40]))      # List concatenation
```

➔ Output

```
30
Good Morning
[10, 20, 30, 40]
```

Operators like + behave polymorphically thereby carrying out addition, concatenation or merging of lists based on the data type of the parameters.

 # Python program to illustrate Polymorphism in Operators

```
print(10 + 20)                      # Integer addition
print("Good " + "Morning")          # String concatenation
print([10, 20] + [30, 40])          # List concatenation
```

➔ Output

```
30
Good Morning
[10, 20, 30, 40]
```

As with other object oriented programming languages, polymorphism in python allows methods in different classes to share the same name but perform different tasks. This is achieved through inheritance and interface design. Polymorphism complements other OOP principles like inheritance (sharing behavior) and

encapsulation (hiding complexity) to create robust and modular applications. It can be seen from the program below that we have not created any common superclass for inheritance or linked the classes together in any form. Still, using polymorphism, we have placed these two different objects namely Faculty and Student into a tuple and iterated through it using a common animal variable as illustrated in the ststement "for person in (Faculty1, Student1, Student2)".

 # Python program to illustrate Polymorphism in Class Methods

```
class Faculty:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"My name is {self.name}. I am {self.age} years old.")

class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"My name is {self.name}. I am {self.age} years old.")

Faculty1 = Faculty("DSK", 55)
Student1 = Student("Karan", 26)
Student2 = Student("Nikita", 30)

for person in (Faculty1, Student1,Student2):
    person.info()
```

➔ Output

```
My name is DSK. I am 55 years old.
My name is Karan. I am 26 years old.
My name is Nikita. I am 30 years old.
```

Abstraction in Python is a way to manage complexity of the program by hiding unnecessary details and showing only essential information. Essentially it means that abstraction is used to hide irrelevant details from the user and show the details that are relevant to the users. It is one of the core principles of Object-Oriented Programming (OOP) paradigm. Abstraction helps in creating a simplified view of objects, making code more manageable and understandable. This property simplifies interaction with objects.

Python also supports abstract and concrete classes.

2.4 ABSTRACT CLASS

An abstract class are not instantiated directly as these serve as a blueprint or template for other classes. It can contain abstract methods that exist without

implementation and concrete methods that are implemented. These abstract classes are created using the abc module and the ABC class. Any subclass that inherits from an abstract class must implement its abstract methods.

A concrete class can be instantiated directly and it inherits methods from an abstract class that are implemented here. A concrete class provides specific implementations for all necessary methods.

```
from abc import ABC, abstractmethod  # Import required modules
class abstraction(ABC):              # Creates Abstract base class
  @abstractmethod                            # Declares abstract method
  def abs_method(self):
    print ("This is abstract method")
    return
  def con_method(self):
    print ("This is concrete method")

class concreteclass(abstraction):            # Creates a child class
  def abs_method(self):
    super().abs_method()
    return

obj = concreteclass()                        # Create an instance of concreteclass
obj.abs_method()
obj.con_method()
```

➔ Output

```
This is abstract method
This is concrete method
```

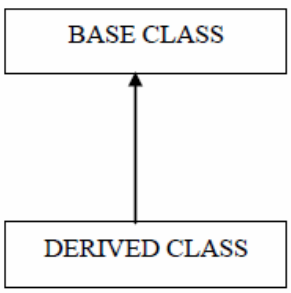
2.5 TYPES OF INHERITANCE

Inheritance can be classified on factors based on the number of base class from which properties have been inherited in the inheritance hierarchy or it could be based on combination of these properties. The major types are:

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

2.5.1 Single Inheritance

When a class is derived from one base class that is when a class inherits properties and methods from only one parent class, then it is termed as single inheritance. This is the simplest form of inheritance and the inheritance diagram is:



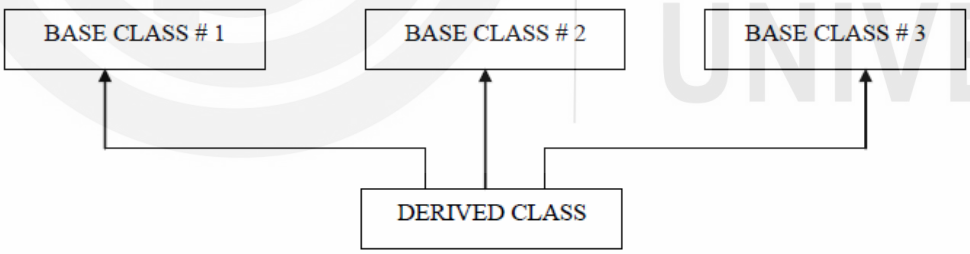
```
# Creating a Parent / Base class
class Parent:
  def ParentMethod(self):
    print ("Johny Calling Method from Parent Class")

# Creating a Child class
class Child(Parent):
  def childMethod(self):
    print ("Johny Calling Method from Child Class")

# Creating Instance of Child
Johny = Child()

# calling method of
Johny.ParentMethod()
print()
# calling method of child class
Johny.childMethod()
```

➔ Output

```
Johny Calling Method from Parent Class

Johny Calling Method from Child Class
```

2.5.2 Multiple Inheritance

When a class is derived from two or more base classes, then a class is termed as multiple inheritance. This kind of inheritance allows one to combine features from several existing classes, thus promoting class reuse. This type of inheritance is suitable in larger programss, where complex hierarchies are needed.




```
class Faculty:
   def Emp_code(self):
      return "Employee Code of FAculty"


class Lab_Attd:
   def Lab_Taken(self):
      return "OOPs Lab"


class Student(Faculty, Lab_Attd):
   pass
```

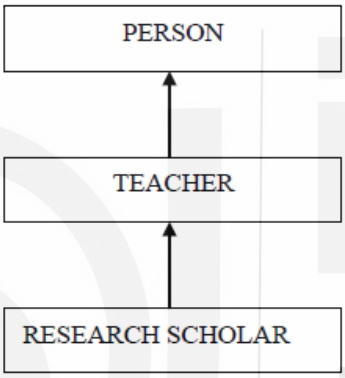
```
# Example usage
student = Student()
print(student.Emp_code())
print(student.Lab_Taken())
```

➔ Output

```
Employee Code of FAculty
OOPs Lab
```

2.5.3 Multilevel Inheritance

When a derived class is obtained from a base class which itself is derived from some other super class, it is termed as multilevel inheritance. Code reuse is maximized in this inheritance by using existing functionalities.




```
# Python program to illustrate Multiple Inheritance
# parent class
class Faculty:
  def Emp_code(self):
    print ("Employee Code of FAculty")

# Child class 'Lab_Attd' Inheriting methods of parent class 'Faculty'
class Lab_Attd(Faculty):
  def Lab_Taken(self):
    print ("OOPs Lab")

# Another child class 'Intern' Inheriting methods of parent class 'Lab_Attd'
class Intern(Lab_Attd):
  def Quiz(self):
    print ("Checks Quiz & Assignment")

# Creating instance of Intern
person = Intern()
```

```
# method calls
person.Emp_code()
print()
person.Lab_Taken()
print()
person.Quiz()
```

➔ Output

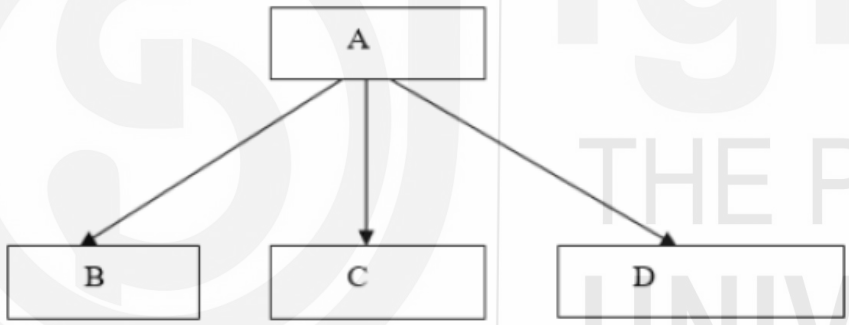
Employee Code of FAculty

OOPs Lab

Checks Quiz & Assignment

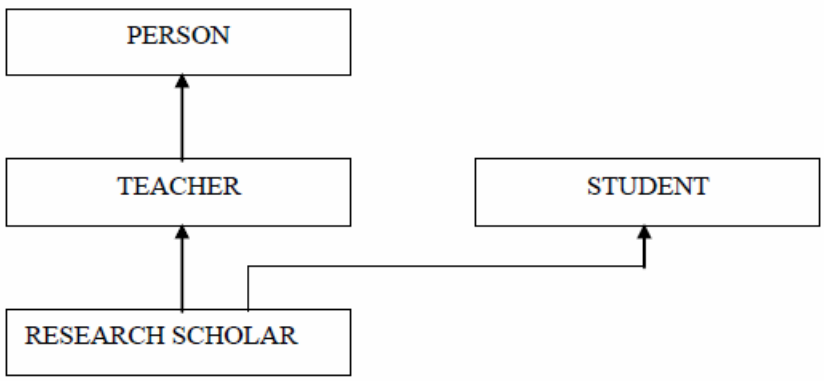
2.5.4 Hierarchical Inheritance

Hierarchical inheritance is a design where multiple child classes inherit from a single parent class. Apparently it shall appear that it promotes strong code reuse, it also leads to tight coupling between the parent and child classes. Hence, any change done in the parent class can have a ripple effect on all child classes that inherit functionalities from these.



2.5.5 Hybrid Inheritance

When a class is derived with a mix of two or more types of inheritance, it is derived as hybrid inheritance.



Here the class Research scholar is derived using multilevel inheritance and single inheritance. Hence this inheritance represents hybrid inheritance. This enables to merge functionalities from multiple parent classes, thus inheriting features from various parents. As an example, suppose we have well defined classes for ICE Car and

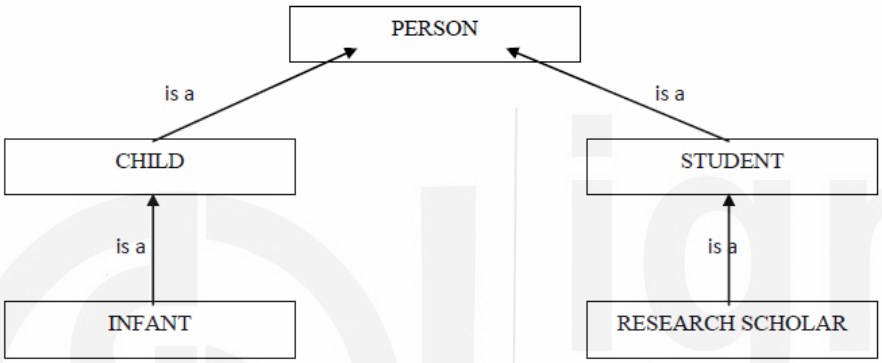
Electric car, one can inherit functionalities from both the classes to create a Hybrid Car class.

2.5.6 Benefits, Costs and Implications of Inheritance

One can use inheritance to create a specialized version from a general purpose base class. Suppose one has a general purpose base class "CYLINDER". This base class could include members such as radius and diameter of cylinder, volume of cylinder. Once a class is derived from this base class, one can include new member variable such as weight of the cylinder or height of cylinder. Thus it is helpful in SPECIALIZATION.

On the other hand, one could inheritance to extend the functionality if a base class, such that the derived class might include some new member functions or new data types. This helps in extending the capability of a class.

Inheritance is also used to represent 'Is a Relationship' among various types of entities.



Among the many reasons, one of the important reason for deriving a sub-class from a base class is to model a new type of object in terms of one or more existing types or to extend the functionality of an existing class i.e. specialization. This specialization is done by adding new member data or member functions.

2.6 NAMESPACE AND OBJECT-ORIENTATION

A namespace is a collection of names where we have a mapping of every name we have defined to corresponding objects in the code. An object can be a variable or a method. Name represents a unique identifier and Space refers to the scope of the name. Python itself maintains a namespace in the form of a Python dictionary. Namespaces are crucial in OOP for organizing and accessing class-level and instance-level variables and methods. As discussed in Chapter 1, each class and object has its own namespace, preventing naming conflicts and ensuring proper access to attributes and methods of the objects. The two basic types of these namespaces are:

- Class Namespace that contains attributes and methods defined within a class that is shared by all instances of the class.
- Instance Namespace contains attributes specific to each instance of a class.

2.7 PYTHON LISTS

A list in Python is a built-in dynamic sized array that automatically grows and shrinks. The list can store various types of data items including another list within a list. This feature is made possible because a list only stores references at contiguous locations and actual items may be stored at different locations. A list can also

contain duplicate items. Accessing any data items in List can be done directly using their subscript or index that starts from 0.

Creating a List in Python

Lists in Python can be created either by using Square Bracket or Using list() Constructor

```
# Creating Python List using Square Bracket []

# List of integers
a = [10, 20, 30, 40, 50]

# List of strings
b = ['C++', 'Java', 'Python']

# Mixed data types
c = [10, 'C++', 100, 'False']

print(a)
print(b)
print(c)
print(c[3])
```

➔ Output

```
[10, 20, 30, 40, 50]
['C++', 'Java', 'Python']
[10, 'C++', 100, 'False']
False
```

 # Creating a Python List Using list() Constructor

```
a = list((10, 'C++', 100, 'False'))

print(a)
```

➔ Output

```
[10, 'C++', 100, 'False']
```

Creating Empty List & Inserting Elements

```
# Initialize an empty list
emptylist = []

# Inserting 5 at index 0
emptylist.insert(0, 5)
print("After Insert(0, 5) operation:", emptylist)
```

➔ Output

```
After Insert(0, 5) operation: [5]
```

 # Accessing Elements of the List by their Indexes

```
print(a[0])  # 5
print(a[1])  # "List"
print(a[2])  # 15
```

```
print(a[3])  # 25
print(a[4])  # True

# Checking for Data Types of list elements

print(type(a[1]))  # str
print(type(a[4]))  # bool
```

➔ Output

```
[5, 'List', 15, 25, True]
5
List
15
25
True
<class 'str'>              # Prints the data type of element at index 1
<class 'bool'>             # Prints the data type of element at index 4
```

 # Python Program to Access last Element of List

```
# List of integers
a = [10, 20, 30, 40, 50]

print(a[-1])
```

➔ Output

```
50
```

Adding Elements into a List using Append & Extend

```
# Python List of integer Numbers
a = [10, 20, 30, 40, 50]

print(a[-1])
a.append(60)
print("List after append(10):", a)
a.extend([70, 80, 90])
print("List after extend(10):", a)
```

➔ Output

```
50
List after append(10): [10, 20, 30, 40, 50, 60]
List after extend(10): [10, 20, 30, 40, 50, 60, 70, 80, 90]
```

2.8 COPY AN OBJECT IN PYTHON USING '='

Python makes use '=' operator to create a copy of an object. One needs to understand that it doesnot create a new object, but it only creates a new variable that shares the reference of the original object. The same is illustrated by a python program below where we have created a list named "original_list" and we pass its object reference to "copied_list" using the '=' operator.

2.8.1 Copy an Object in Python using Copy( ) Method

Python lists are useful data structures used to store a collection of items. The copy() list method is important when working with data since it helps preserve the original list while performing operations on the duplicated one. This prevents from any untoward data loss.

The copy() method is a built-in method in Python that is used to create a shallow copy of the list. The copy() method has been made available only from Python 3.3 version onwards. This method copies only the top-level list structure. The syntax of the copy() method is shown below.

2.8.2 Copy a List in Python

```
# Define the original list ofElements
original_list = [10, 20, 30, 40]

# Creating a Shallow copy of the Original List
copied_list = original_list.copy()

# Modifying the first element of the Copied List
copied_list[0] = 99

print(original_list)
print(copied_list)
```

➔ Output

```
[10, 20, 30, 40]
[99, 20, 30, 40]
```

Shallow and Deep Copying

A shallow copy in Python doesn't create a copy of objects, instead it just copies the reference of the objects. A deep copy creates a new object and recursively adds the copies of objects present in the original elements. Hence a new object with all the list elements is created in the deep copying operation.

The Copy Module of Python

One has to make use the copy module of Python for shallow and deep copy operations. Suppose, you need to copy the compound list say elementlist. Then one makes use of the following:

```
import copy
copy.copy(elementlist)          # Shallow Copy
copy.deepcopy(elementlist)          # Deep Copy
```

Creating a Shallow Copy of the List

```
import copy

original_list = [[10, 20, 30], [40, 50, 60]]
copied_list = copy.copy(original_list)

print("Original list:", original_list)
print("Copy of Original list:", copied_list)
```

➔ Output

```
Original list: [[10, 20, 30], [40, 50, 60]]
Copy of Original list: [[10, 20, 30], [40, 50, 60]]
```

Now if we change the third element of the copied_list to 100, we obtain the following output.

```
copied_list[0][2] = 100

print("Original list:", original_list)
print("Copy of Original list:", copied_list)
```

➔ Output

```
Original list: [[10, 20, 100], [40, 50, 60]]
Copy of Original list: [[10, 20, 100], [40, 50, 60]]
```

It is evident from the above piece of code that by changing a nested / child element of the copied_list, the same affect gets reflected in the original_list. This is because the object resident in the dynamic memory is referenced by the pointers by both the original_list and the copied_list. Hence any change made in the copied_list actually changes the value of the object resident in the dynamic memory or heap. In order to avoid this, we need to make use of deep copy.

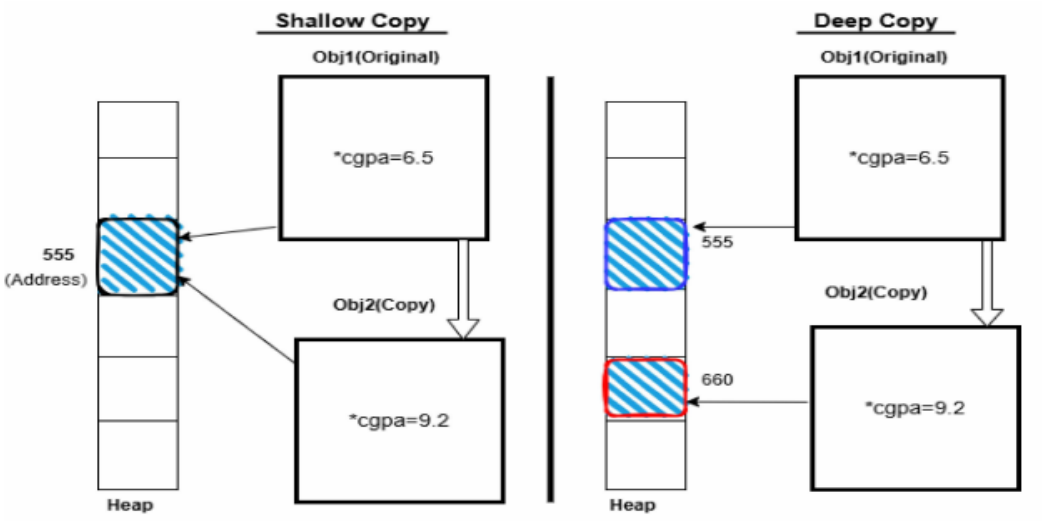
2.8.3 Deep Copy

A deep copy carries out a recursive copying process of the original collection object creates a copy of the object, and it does so by populating copies of the child objects found in the original. That means that the new collection object (copied_list) and its elements are independent of the original (original_list).

```
import copy

original_list = [[10, 20, 30], [40, 50, 60]]
copied_list = copy.deepcopy(original_list)

print("Original list:", original_list)
print("Deep Copy of Original list:", copied_list)

print()

copied_list[0][2] = 100

print("Original list:", original_list)
print("Deep Copy of Original list:", copied_list)
```

➔ Output

```
Original list: [[10, 20, 30], [40, 50, 60]]
Deep Copy of Original list: [[10, 20, 30], [40, 50, 60]]

Original list: [[10, 20, 30], [40, 50, 60]]
Deep Copy of Original list: [[10, 20, 100], [40, 50, 60]]
```



The concepts of shallow and deep copying have been illustrated in the figure. Now it should be amply clear that the Shallow copies are efficient and faster because it uses less heap memory but can lead to unintended changes. Deep copies use more memory and processing time but ensure the copied list is fully independent and any change made in the copy shall not affect the original.

However one needs to exercise certain cautions. Recursive objects also referred to as compound objects that, directly or indirectly, contain a reference to themselves may cause a recursive loop. Also, since deep copy copies everything it may copy too much, such as data which is meant to be shared between the copies of an object.

2.9 SUMMARY

Object-oriented programming paradigm provides complete insulation from internal architecture and operations. It provides a very high level of abstraction in problem solving. Programs are designed with a focus on data around which the code is built and this complete entity is known as object. This is a more realistic way of representing the world in general. Python as Object-Oriented Programming Language is to write structured programs by creating "objects," that encapsulate data or attributes and functions or methods that operate on that data. This improves the program structure and enhances code reusability. Major types of Inheritance supported by Python are Single inheritance, Multiple Inheritance, Multilevel Inheritance, Hierarchical Inheritance and Hybrid Inheritance.

Polymorphism allows methods carry the same method name as in super class but different functionality based on the object's context. This is achieved through method overriding or overloading. Polymorphism facilitates dynamic method usage, where the specific method to be executed is determined at runtime based on the object's actual type. Abstraction helps to manage complexity of the program by hiding unnecessary details and showing only essential information to the programmer.

An abstract class are not instantiated directly as these serve as a blueprint or template for other classes. Abstract methods exist without its implementation and only the concrete methods that are implemented. A concrete class can be instantiated directly and it inherits methods from an abstract class that are implemented here. Among the many reasons, one of the important reasons for deriving a sub-class from a base class is to model a new type of object in terms of one or more existing types or to extend the functionality of an existing class i.e. specialization.

Namespaces are crucial in OOP for organizing and accessing class-level and instance-

level variables and methods. As discussed in Chapter 1, each class and object has its own namespace, preventing naming conflicts and ensuring proper access to attributes and methods of the objects. The two basic types of these namespaces are Class Namespace that contains attributes and methods defined within a class that is shared by all instances of the class and Instance Namespace contains attributes specific to each instance of a class.

A list in Python is a built-in dynamic sized array that automatically grows and shrinks. The list can store various types of data items including another list within a list. A list can also contain duplicate items. Accessing any data items in List can be done directly using their subscript or index that starts from 0.

Python makes use '=' operator to create a copy of an object. One needs to understand that it doesnot create a new object, but it only creates a new variable that shares the reference of the original object. A shallow copy in Python doesn't create a copy of objects, instead it just copies the reference of the objects. A deep copy creates a new object and recursively adds the copies of objects present in the original elements. Deep copies use more memory and processing time but ensure the copied list is fully independent and any change made in the copy shall not affect the original.

Check your Progress

Q1. How can one concatenate two lists in Python?

Q2. **What is pass in Python coding?**

Q3. What are Modules and Packages in Python?

Q4. What is the difference between a shallow copy and a deep copy?

Q5. **Does Python support multiple Inheritance?**

Q6. **What is the significance of Polymorphism in Python?**

Q7. **What is __init__() in Python and how does self play a role in it?**

Q8. What is init method in python?

Q9. Is it possible to call parent class without creating its instance?

Q10. What is the significance of inheritance in python?

Q11. Why does one use else in try/except construct in Python?

2.10 FURTHER READINGS

1. Python Essentials You Always Wanted to Know: Beginner's Guide to Python Programming, Data Structures, Data Analytics with Hands-On Coding Exercises by Shawn Peters and Vibrant Publishers

2. Python Programming : Beginners Guide Paperback
by Sundarrajan M (Author), Mani Deepak Choudhry (Author), Jeevanandham S (Author), Akshya Jothi (Author)

3. The easy basics of Python: Getting started with object-oriented programming using Python by Nicolas Snake (ebook)

Reference Websites

https://pythoninstitute.org/python-essentials-1

https://pythoninstitute.org/python-essentials-2

ignou
THE PEOPLE'S
UNIVERSITY

UNIT 3 ALGORITHM ANALYSIS

Structure

3.0 Introduction
3.1 Objectives
3.2 Some Additional Objectives
3.3 Experimental Studies
3.4 The Seven Functions Used in This Book
3.5 Asymptotic Analysis
3.6 Simple Justification Techniques
3.7 Summary
3.8 Solutions / Answers
3.9 Further Readings

3.0 INTRODUCTION

In the previous unit, we have discussed fundamental data structures and their implementations. Now, we understand that storing and manipulating data is only half the battle in computer science. The critical question that arises is: "How efficient is our solution?" Two different algorithms may solve the same problem, but one may be significantly faster than the other, especially when dealing with large datasets.

Algorithm analysis is the process of determining the amount of computing resources (such as time and memory) that an algorithm requires. This unit will provide you with a comprehensive understanding of how to measure, compare, and optimize algorithm performance. We will explore various analytical techniques that help us predict how an algorithm will behave as the input size grows, allowing us to make informed decisions about which data structure and algorithm to choose for a given problem.

Understanding algorithm analysis is not merely an academic exercise. It is a practical necessity. Consider a scenario where you are designing a system to process millions of financial transactions. An algorithm that performs acceptably on 1,000 transactions may become completely unusable on 1,000,000 transactions if its time complexity is poor. By mastering the concepts in this unit, you will be able to identify algorithmic bottlenecks and design solutions that scale efficiently.

3.1 OBJECTIVES

After going through this unit, you will be able to:

- Understand the importance and significance of algorithm analysis
- Conduct experimental studies to measure algorithm performance
- Identify and apply the seven fundamental functions used in algorithm analysis
- Apply asymptotic notation (Big O, Omega, and Theta) to analyze algorithm complexity
- Distinguish between best-case, average-case, and worst-case scenarios
- Use simple justification techniques to prove algorithm correctness and efficiency
- Make informed decisions about algorithm selection based on performance analysis
- Analyze recursive algorithms and determine their time and space complexity

3.2 SOME ADDITIONAL OBJECTIVES

The overarching goals of this unit are to equip you with the analytical tools and theoretical foundation necessary to:

1. **Quantify Algorithm Performance**: Develop the ability to measure and express how algorithms scale with increasing input sizes.

2. **Compare Algorithms Objectively**: Learn methods to objectively compare different algorithms solving the same problem and identify which is most suitable for specific scenarios.
3. **Predict Real-World Performance**: Use theoretical analysis to predict how algorithms will behave in production environments without running extensive experiments.
4. **Optimize for Scale**: Understand the implications of choosing different data structures and algorithmic approaches as data volumes grow from thousands to millions to billions of elements.
5. **Make Engineering Decisions**: Develop the judgment to make informed trade-offs between simplicity, correctness, and performance.
6. **Understand Fundamental Limits**: Recognize that some problems have inherent complexity limits that no algorithm can overcome, and learn to identify when you have reached those limits.

3.3 EXPERIMENTAL STUDIES

Experimental studies involve running algorithms on actual computers and measuring their performance. This empirical approach provides concrete data about how algorithms behave in practice.

3.3.1 Methodology for Experimental Studies

Step 1: Algorithm Implementation Implement the algorithm in a suitable programming language. The implementation should be as efficient as possible to avoid measuring the overhead of a poor implementation.
Step 2: Test Case Generation Create test cases with varying input sizes. Typically, we start with small inputs and progressively increase the size, often doubling the size at each step.
Step 3: Performance Measurement Execute the algorithm on each test case and measure:

- **Execution Time**: Wall-clock time taken for the algorithm to complete
- **Memory Usage**: Peak memory consumed during execution
- **Operations Count**: Number of primitive operations performed

Step 4: Data Collection and Analysis Record results in a structured format and analyze how performance changes with input size.

3.3.2 Practical Example: Analyzing Sorting Algorithms

```
import time
import random

def bubble_sort(arr):
    """
    Bubble Sort Algorithm
    Time Complexity: O(n²)
    """
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

def quick_sort(arr):
    """
```

```
    Quick Sort Algorithm
    Time Complexity: O(n log n) average case
    """
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

# Experimental study
def conduct_experiment():
    """
    Conduct experimental study comparing sorting algorithms
    """
    input_sizes = [100, 500, 1000, 5000, 10000]
    results = {
        'bubble_sort': [],
        'quick_sort': []
    }

    for size in input_sizes:
        # Generate random test data
        test_data_bubble = [random.randint(1, 1000) for _ in range(size)]
        test_data_quick = test_data_bubble.copy()

        # Measure bubble sort
        start_time = time.time()
        bubble_sort(test_data_bubble)
        bubble_time = time.time() - start_time
        results['bubble_sort'].append(bubble_time)

        # Measure quick sort
        start_time = time.time()
        quick_sort(test_data_quick)
        quick_time = time.time() - start_time
        results['quick_sort'].append(quick_time)

        print(f"Input Size: {size}")
        print(f"  Bubble Sort: {bubble_time:.6f} seconds")
        print(f"  Quick Sort: {quick_time:.6f} seconds")
        print()

    return results

# Execute experiment
# conduct_experiment()
```

3.3.3 Observations from Experimental Studies

When we conduct this experiment, we observe:

- For small inputs (< 100 elements), both algorithms complete almost instantaneously
- As input size increases, bubble sort's execution time grows much faster than quick sort

- With 10,000 elements, bubble sort may take several seconds while quick sort completes in milliseconds
- The performance gap widens as input size increases

Limitation of Experimental Studies: While experimental studies provide valuable insights, they have limitations:

- Results are specific to the hardware used
- Only provides data for tested input sizes; extrapolation is needed for larger inputs
- Implementation details can significantly affect results
- Testing on all possible inputs is impractical

This is where theoretical analysis becomes indispensable.

3.4 THE SEVEN FUNCTIONS USED IN THIS BOOK

In algorithm analysis, we use several fundamental mathematical functions to characterize algorithm complexity. These functions describe how the running time or space requirement grows as the input size increases.

3.4.1 Constant Function: f(n) = 1

Definition: The algorithm takes the same time regardless of input size.
Characteristics:

- The most desirable complexity class
- Running time does not depend on the size of the input
- Examples: accessing an array element by index, performing arithmetic operations

Code Example:

```
def get_first_element(arr):
    """
    This function always takes constant time
    Time Complexity: O(1)
    """
    if len(arr) > 0:
        return arr[0]
    return None

# Analysis:
# - Accessing arr[0] is a single operation: O(1)
# - The if condition check is O(1)
# - Total time is constant, regardless of array size
```

Visual Representation:

```
Time
 |
 |------------ (constant)
 |
 |_____________________________
 0          Input Size (n)
```

3.4.2 Logarithmic Function: f(n) = log n

Definition: The algorithm divides the input repeatedly by a constant factor.
Characteristics:

- Highly efficient even for very large inputs

- Grows very slowly as input size increases
- Common in divide-and-conquer algorithms like binary search
- Base of logarithm: typically 2 in computer science (log n)

Code Example:

```
def binary_search(arr, target):
    """
    Binary Search Algorithm
    Time Complexity: O(log n)

    Precondition: arr must be sorted
    """
    left, right = 0, len(arr) - 1
    comparisons = 0

    while left <= right:
        comparisons += 1
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid, comparisons
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1, comparisons

# Analysis:
# - Each iteration eliminates half of remaining elements
# - Maximum iterations: log (n)
# - For n = 1,000,000, maximum comparisons ≈ 20

# Example execution:
arr = sorted([3, 7, 2, 9, 1, 5, 8, 4, 6])
result = binary_search(arr, 5)
print(f"Found at index: {result[0]}, Comparisons: {result[1]}")
```

Growth Comparison:

n	log (n)
10	3.3
100	6.6
1,000	9.9
10,000	13.3
1,000,000	19.9

3.4.3 Linear Function: f(n) = n

Definition: The algorithm processes each input element once.
Characteristics:

- Common in simple search and traversal operations
- Acceptable for most practical applications
- Linear relationship between input size and execution time
- Doubling input size doubles execution time

Code Example:

```
def linear_search(arr, target):
    """
    Linear Search Algorithm
    Time Complexity: O(n)
    """
    comparisons = 0

    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == target:
            return i, comparisons

    return -1, comparisons

# Analysis:
# - Worst case: traverse entire array → n comparisons
# - Average case: traverse half array → n/2 comparisons
# - Still considered O(n) as constants are ignored

# Comparison with Binary Search:
arr = list(range(1, 1000001))  # 1 million elements

# Linear search for 999,999:
lin_result = linear_search(arr, 999999)
print(f"Linear Search: {lin_result[1]} comparisons")

# Binary search for same element:
bin_result = binary_search(arr, 999999)
print(f"Binary Search: {bin_result[1]} comparisons")
```

Visual Comparison: O(log n) vs O(n)

```
Time
 |                O(n) - Linear
 |               /
 |              /
 |     O(log n) ---/
 |    -/
 |____/____________________________
 0        Input Size (n)
```

3.4.4 Linear Logarithmic Function: f(n) = n log n

Definition: The algorithm divides the input and processes all elements at each level.
Characteristics:

- Efficiency class of optimal sorting algorithms
- Between O(n) and O(n²)
- Common in divide-and-conquer algorithms like merge sort
- Grows faster than linear but much slower than quadratic

Code Example:

```
def merge_sort(arr):
    """
    Merge Sort Algorithm
    Time Complexity: O(n log n)
```

```
    """
    if len(arr) <= 1:
        return arr

    # Divide: O(1)
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]

    # Conquer: T(n/2) + T(n/2)
    left_sorted = merge_sort(left)
    right_sorted = merge_sort(right)

    # Combine: O(n)
    return merge(left_sorted, right_sorted)

def merge(left, right):
    """
    Merge two sorted arrays
    Time Complexity: O(n)
    """
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
```

```
# Analysis:
# Recurrence: T(n) = 2·T(n/2) + O(n)
# By Master Theorem: T(n) = O(n log n)
# - Dividing: log n levels
# - Each level processes n elements
# - Total: n × log n operations
```

3.4.5 Quadratic Function: $f(n) = n^2$

Definition: The algorithm contains nested loops, each iterating over the input.
Characteristics:

- Acceptable for small inputs (< 10,000 elements)
- Becomes problematic for large datasets
- Common in elementary sorting algorithms
- Doubling input size quadruples execution time

Code Example:

```
def bubble_sort_detailed(arr):
    """
    Bubble Sort Algorithm
    Time Complexity: O(n²)
    """
    n = len(arr)
    comparisons = 0
    swaps = 0

    for i in range(n):
        for j in range(0, n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1

    return arr, comparisons, swaps
```

```
# Analysis:
# Outer loop: n iterations
# Inner loop: average n/2 iterations
# Total: n × (n/2) = n²/2 comparisons
# Still O(n²) - constants ignored

# Example: Impact on different input sizes
for size in [100, 1000, 10000]:
    arr = list(range(size, 0, -1))  # Worst case: reverse sorted
    _, comps, swaps = bubble_sort_detailed(arr.copy())
    print(f"Size {size}: {comps} comparisons, {swaps} swaps")
```

Growth Pattern:

n	n^2
10	100
100	10,000
1,000	1,000,000
10,000	100,000,000

3.4.6 Cubic Function: $f(n) = n^3$

Definition: The algorithm contains three nested loops, each iterating over the input.

Characteristics:

- Rarely acceptable for practical applications
- Used in some matrix operations and advanced algorithms
- Even moderate input sizes become problematic
- Doubling input size increases execution time by 8 times

Code Example:

```
def matrix_multiplication(A, B):
    """
    Standard Matrix Multiplication
    Time Complexity: O(n³) for n×n matrices
    """
    n = len(A)
    C = [[0] * n for _ in range(n)]
    operations = 0

    for i in range(n):
```

```
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
                operations += 1

    return C, operations

# Analysis:
# Three nested loops: each iterating n times
# Total operations: n × n × n = n³

# Performance impact:
import time

for size in [10, 50, 100]:
    A = [[i + j for j in range(size)] for i in range(size)]
    B = [[i * j for j in range(size)] for i in range(size)]

    start = time.time()
    _, ops = matrix_multiplication(A, B)
    elapsed = time.time() - start

    print(f"Size {size}×{size}: {ops} operations, {elapsed:.6f} seconds")
```

3.4.7 Exponential Function: f(n) = 2^n

Definition: The algorithm's running time doubles with each additional input element.
Characteristics:

- Practically infeasible for input sizes beyond 40-50 elements
- Common in brute-force algorithms and certain recursive problems
- Represents truly hard problems (NP-complete problems)
- Only viable approach: approximation or heuristic algorithms

Code Example:

```
def fibonacci_exponential(n):
    """
    Recursive Fibonacci Computation
    Time Complexity: O(2^n) - exponential
    """
    if n <= 1:
        return n
    return fibonacci_exponential(n - 1) + fibonacci_exponential(n - 2)

# This demonstrates why exponential algorithms are impractical:
# fibonacci_exponential(40) requires approximately 2^40 ≈ 1 trillion calls!

# Better approach: Dynamic Programming - O(n)
def fibonacci_dp(n):
    """
    Fibonacci using Dynamic Programming
    Time Complexity: O(n) - linear
    """
    if n <= 1:
        return n

    dp = [0] * (n + 1)
```

```
    dp[0], dp[1] = 0, 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]

# Comparison:
import time

# Exponential approach (only for small n)
for n in [30, 32, 34, 36]:
    start = time.time()
    result = fibonacci_exponential(n)
    elapsed = time.time() - start
    print(f"fibonacci_exponential({n}): {result}, Time: {elapsed:.6f}s")

# DP approach (much faster)
for n in [30, 100, 200, 500]:
    start = time.time()
    result = fibonacci_dp(n)
    elapsed = time.time() - start
    print(f"fibonacci_dp({n}): {result}, Time: {elapsed:.6f}s")
```

Comparison Table of Seven Functions:

Input Size	1	log n	n	n log n	n^2	n^3	2^n
10	1	3	10	33	100	1,000	1,024
100	1	7	100	664	10K	1M	2^100
1,000	1	10	1K	10K	1M	1B	2^1000
10,000	1	13	10K	132K	100M	1B	2^10000

3.5 ASYMPTOTIC ANALYSIS

Asymptotic analysis allows us to characterize algorithm performance without relying on experimental measurements or specific hardware configurations. It focuses on how the algorithm behaves as the input size approaches infinity.

3.5.1 Big-O Notation (Upper Bound)

Definition: f(n) = O(g(n)) means there exist positive constants c and n such that f(n) ≤ c·g(n) for all n ≥ n .
Interpretation: O(g(n)) represents the worst-case scenario - an upper bound on the running time. The algorithm will never take longer than this.
Common Usage:

- Describes the maximum time an algorithm will take
- Most commonly used notation in practice
- Conservative estimate of algorithm behavior

Examples:

```
def example1(arr):
    """
    Time Complexity: O(n)

    Explanation:
    - One loop through array: n iterations
```

```
    - Inside loop: constant time operations (O(1))
    - Total: n × O(1) = O(n)
    """
    total = 0
    for i in arr:
        total += i  # O(1)
    return total

def example2(arr):
    """
    Time Complexity: O(n²)

    Explanation:
    - Outer loop: n iterations
    - Inner loop: n iterations for each outer iteration
    - Total: n × n = O(n²)
    """
    for i in range(len(arr)):
        for j in range(len(arr)):
            if arr[i] == arr[j] and i != j:
                print(f"Found duplicate at positions {i} and {j}")

def example3(arr):
    """
    Time Complexity: O(n + m) where arr has n elements and
    we perform m additional operations
    """
    # O(n) - traverse array once
    for i in arr:
        print(i)

    # O(m) - perform m additional operations
    for j in range(m):
        print(j)

    # Total: O(n + m)

def example4(arr):
    """
    Time Complexity: O(1)

    Explanation:
    - Creates a list of fixed size (10 elements)
    - No loops depending on input size
    - Constant operations regardless of arr size
    """
    new_list = [0] * 10
    return new_list
```

Rules for Big-O Analysis:

Drop Constants: O(2n) = O(n), O(5n²) = O(n²)

```
# Both are O(n) despite different constant factors
def function1(arr):
    for i in range(len(arr)):  # n iterations
        print(i)

def function2(arr):
```

```
    for i in range(len(arr)):  # n iterations
        print(i)
    for i in range(len(arr)):  # n iterations
        print(i)
# function2 is O(2n) = O(n)
```

Drop Non-Dominant Terms: $O(n^2 + n) = O(n^2)$

```
def analyze_complexity(n):
    """
    Time Complexity: O(n² + n) = O(n²)
    The n² term dominates n for large n
    """
    for i in range(n):          # n² iterations total
        for j in range(n):
            print(i, j)

    for i in range(n):          # n iterations
        print(i)
```

Multiply for Nested Structures: Sequential operations add, nested operations multiply

```
# O(n) + O(n) = O(n)
def sequential(arr):
    for i in arr:
        print(i)
    for i in arr:
        print(i)

# O(n) × O(n) = O(n²)
def nested(arr):
    for i in arr:
        for j in arr:
            print(i, j)
```

3.5.2 Omega Notation (Lower Bound)

Definition: $f(n) = \Omega(g(n))$ means there exist positive constants c and n such that $f(n) \geq c \cdot g(n)$ for all $n \geq n$.

Interpretation: $\Omega(g(n))$ represents the best-case scenario - a lower bound on the running time. The algorithm will never be faster than this.

Code Example:

```
def linear_search_omega(arr, target):
    """
    Linear Search
    - Best case: O(1) - element found at first position
    - Omega (lower bound): Ω(1) - at least one comparison
    - Worst case: O(n) - element at last position or not present
    """
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Demonstration
arr = [5, 3, 7, 1, 9, 2]

# Best case: Ω(1)
result1 = linear_search_omega(arr, 5)  # Found immediately
```

```
# Worst case: O(n)
result2 = linear_search_omega(arr, 100)  # Not found, searched entire array

# Average case: Between Ω(1) and O(n)
result3 = linear_search_omega(arr, 1)   # Found at position 3
```

3.5.3 Theta Notation (Tight Bound)

Definition: f(n) = Θ(g(n)) means f(n) = O(g(n)) AND f(n) = Ω(g(n))). Both upper and lower bounds exist and are the same order of magnitude.
Interpretation: Θ(g(n)) provides a tight bound - the actual running time is within a constant factor of g(n).

Code Example:

```
def example_theta(arr):
    """
    Time Complexity: Θ(n)

    Explanation:
    - In all cases (best, average, worst), the algorithm
      processes each element exactly once
    - Upper bound: O(n)
    - Lower bound: Ω(n)
    - Therefore: Θ(n)
    """
    total = 0
    for i in arr:
        total += i  # Always executed n times
    return total

# This function has tight bound Θ(n)
# because it always executes the loop n times,
# regardless of the input values
```

3.5.4 Asymptotic Comparison

Complexity Order (Best to Worst):
1. O(1) - Constant
2. O(log n) - Logarithmic
3. O(n) - Linear
4. O(n log n) - Linear-Logarithmic
5. $O(n^2)$ - Quadratic
6. $O(n^3)$ - Cubic
7. O(2^n) - Exponential
8. O(n!) - Factorial

For n = 1,000:
O(1) ≈ 1
O(log n) ≈ 10
O(n) ≈ 1,000
O(n log n)≈ 10,000
$O(n^2)$ ≈ 1,000,000
$O(n^3)$ ≈ 1,000,000,000
O(2^n) ≈ 10^301 (astronomically large)

3.5.5 Master Theorem for Recurrence Relations

The Master Theorem provides a quick way to analyze recursive algorithms without solving recurrence relations directly.

Theorem: For a recurrence $T(n) = a \cdot T(n/b) + f(n)$ where $a \geq 1$ and $b > 1$:

1. If $f(n) = O(n^{(\log_b(a) - \varepsilon)})$ for some $\varepsilon > 0$, then $\mathbf{T(n) = \Theta(n^{(\log_b a)})}$
2. If $f(n) = \Theta(n^{(\log_b a)})$, then $\mathbf{T(n) = \Theta(n^{(\log_b a)} \cdot \log n)}$
3. If $f(n) = \Omega(n^{(\log_b(a) + \varepsilon)})$ for some $\varepsilon > 0$, and $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$, then $\mathbf{T(n) = \Theta(f(n))}$

Code Examples:

```
# Example 1: Merge Sort
# T(n) = 2·T(n/2) + O(n)
# a=2, b=2, f(n)=n
# log_b(a) = log_2(2) = 1
# f(n) = n = n^1 = Θ(n^1)
# Case 2 applies: T(n) = Θ(n · log n)

def merge_sort_analysis():
    """
    Master Theorem Analysis:
    T(n) = 2·T(n/2) + O(n)
    a = 2, b = 2, f(n) = n
    n^(log_2 2) = n^1 = n
    f(n) = Θ(n), so Case 2 applies
    Result: T(n) = Θ(n log n)
    """
    pass

# Example 2: Binary Search
# T(n) = 1·T(n/2) + O(1)
# a=1, b=2, f(n)=1
# log_b(a) = log_2(1) = 0
# n^0 = 1 = Θ(1)
# Case 2 applies: T(n) = Θ(1 · log n) = Θ(log n)

def binary_search_analysis():
    """
    Master Theorem Analysis:
    T(n) = 1·T(n/2) + O(1)
    a = 1, b = 2, f(n) = 1
    n^(log_2 1) = n^0 = 1
    f(n) = Θ(1), so Case 2 applies
    Result: T(n) = Θ(log n)
    """
    pass

# Example 3: Strassen's Matrix Multiplication
# T(n) = 7·T(n/2) + O(n²)
# a=7, b=2, f(n)=n²
# log_b(a) = log_2(7) ≈ 2.807
# n^2.807 > n², so f(n) = O(n^2.807 - ε)
# Case 1 applies: T(n) = Θ(n^2.807)

def strassen_analysis():
```

```
    """
    Master Theorem Analysis:
    T(n) = 7·T(n/2) + O(n²)
    a = 7, b = 2, f(n) = n²
    n^(log_2 7) ≈ n^2.807
    f(n) = n² = O(n^(2.807 - 0.807))
    Case 1 applies
    Result: T(n) = Θ(n^2.807) (improvement over standard O(n³))
    """
    pass
```

3.6 SIMPLE JUSTIFICATION TECHNIQUES

To rigorously establish the correctness and efficiency of algorithms, we employ several justification techniques.

3.6.1 Loop Invariants

A loop invariant is a logical statement that:

- Is true before the loop begins (initialization)
- Remains true at each iteration (maintenance)
- Becomes our conclusion when the loop terminates (termination)

Example: Insertion Sort with Loop Invariants

```
def insertion_sort_with_invariant(arr):
    """
    Insertion Sort Algorithm with Loop Invariant Analysis

    Loop Invariant: At the start of each iteration i, the subarray
    arr[0:i] contains the first i elements of the original array,
    sorted in ascending order.

    Proof:
    1. Initialization (i=1): arr[0:1] has one element, trivially sorted ✓
    2. Maintenance: If arr[0:i] is sorted, we insert arr[i] into the
       correct position in arr[0:i], maintaining the sorted property ✓
    3. Termination: When i = n, arr[0:n] is sorted, which is our goal ✓
    """
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Invariant: arr[j+1:i] are all greater than key
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr

# Verification
test_array = [64, 34, 25, 12, 22, 11, 90]
result = insertion_sort_with_invariant(test_array)
print(f"Sorted array: {result}")
```

```
# Expected output: [11, 12, 22, 25, 34, 64, 90]
```

3.6.2 Mathematical Induction

Mathematical induction is a powerful proof technique that establishes the correctness of algorithms, particularly those involving recursive structures or operations on sequences.

Principle of Mathematical Induction: To prove a statement P(n) is true for all positive integers n:

1. **Base Case:** Show that P(1) is true
2. **Inductive Step:** Assume P(k) is true for some k, and prove P(k+1) is true
3. **Conclusion:** Then P(n) is true for all positive integers n

Example 1: Proving Summation Formula

```
def sum_of_first_n_natural_numbers(n):
    """
    Calculate sum: 1 + 2 + 3 + ... + n

    Claim: sum(n) = n(n+1)/2

    Proof by Mathematical Induction:

    Base Case (n=1):
    sum(1) = 1
    Formula: 1(1+1)/2 = 1
    ✓ Base case holds

    Inductive Step:
    Assume sum(k) = k(k+1)/2 is true
    Prove sum(k+1) = (k+1)(k+2)/2

    sum(k+1) = sum(k) + (k+1)
             = k(k+1)/2 + (k+1)
             = [k(k+1) + 2(k+1)]/2
             = [(k+1)(k+2)]/2
    ✓ Inductive step holds

    Therefore, sum(n) = n(n+1)/2 for all positive integers n
    """
    # Implementation
    return n * (n + 1) // 2

# Verification
for n in range(1, 11):
    iterative_sum = sum(range(1, n + 1))
    formula_result = sum_of_first_n_natural_numbers(n)
    print(f"n={n}: Direct sum={iterative_sum}, Formula={formula_result},
Match={iterative_sum == formula_result}")
```

Example 2: Proving Recursive Algorithm Correctness

```
def factorial_recursive(n):
    """
    Calculate factorial: n! = n × (n-1) × (n-2) × ... × 1
```

```
    Claim: factorial_recursive(n) correctly computes n!

    Proof by Mathematical Induction:

    Base Case (n=0):
    factorial_recursive(0) returns 1
    By definition, 0! = 1
    ✓ Base case holds

    Inductive Step:
    Assume factorial_recursive(k) correctly computes k!
    Prove factorial_recursive(k+1) correctly computes (k+1)!

    factorial_recursive(k+1) = (k+1) × factorial_recursive(k)
                             = (k+1) × k!    [by inductive hypothesis]
                             = (k+1)!
    ✓ Inductive step holds

    Therefore, factorial_recursive(n) correctly computes n!
    for all non-negative integers n
    """
    if n == 0:
        return 1
    else:
        return n * factorial_recursive(n - 1)

# Verification
for n in range(0, 8):
    import math
    computed = factorial_recursive(n)
    expected = math.factorial(n)
    print(f"{n}! = {computed}, Expected = {expected}, Match = {computed ==
expected}")
```

Example 3: Proving Array Operations

```
def array_sum_proof(arr, n=None):
    """
    Claim: Summing all n elements of an array produces the correct total

    Proof by Mathematical Induction:

    Base Case (n=1):
    Sum of single element = that element
    ✓ Correct

    Inductive Step:
    Assume sum of first k elements is correct: S(k)
    When we add element k+1: S(k+1) = S(k) + arr[k+1]
    Since S(k) is correct and we add arr[k+1], S(k+1) is correct
    ✓ Inductive step holds

    Therefore, summing n elements correctly computes the sum
    """
    if n is None:
        n = len(arr)
```

```
    if n == 1:
        return arr[0]
    else:
        return array_sum_proof(arr, n - 1) + arr[n - 1]

# Verification
test_arrays = [
    [5],
    [1, 2, 3],
    [10, 20, 30, 40],
    [-5, 10, -3, 8, 2]
]

for arr in test_arrays:
    result = array_sum_proof(arr)
    expected = sum(arr)
    print(f"Array {arr}: Computed={result}, Expected={expected}, Correct={result ==
expected}")
```

Example 4: Strong Induction

Strong induction is useful when proving P(n+1) requires knowledge of multiple previous cases, not just P(n).

```
def minimum_coins_payment(amount, coins=[1, 5, 10, 25]):
    """
    Claim: Any amount can be made using coins [1, 5, 10, 25]

    Proof by Strong Induction:

    Base Case (amount=1):
    1 = 1×(coin of 1)
    ✓ Base case holds

    Inductive Step:
    Assume all amounts 1 to k can be made with given coins
    Prove amount k+1 can be made:

    If k+1 ≤ 4: k+1 = k+1×(coin of 1)
    If k+1 ≥ 5: (k+1) - 5 ≤ k, so by inductive hypothesis,
            (k+1) - 5 can be made, thus k+1 can be made
            by adding one 5-cent coin

    ✓ Inductive step holds

    Therefore, any positive amount can be made
    """

    # Dynamic programming solution
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount]
```

```
# Verification
for amount in range(1, 26):
    min_coins = minimum_coins_payment(amount)
    print(f"Amount {amount}: Minimum coins needed = {min_coins}")
```

Example 2: Proving Recursive Algorithm Correctness

```
def factorial_recursive(n):
    """
    Calculate factorial: n! = n × (n-1) × (n-2) × ... × 1

    Claim: factorial_recursive(n) correctly computes n!

    Proof by Mathematical Induction:

    Base Case (n=0):
    factorial_recursive(0) returns 1
    By definition, 0! = 1
    ✓ Base case holds

    Inductive Step:
    Assume factorial_recursive(k) correctly computes k!
    Prove factorial_recursive(k+1) correctly computes (k+1)!

    factorial_recursive(k+1) = (k+1) × factorial_recursive(k)
                             = (k+1) × k!    [by inductive hypothesis]
                             = (k+1)!
    ✓ Inductive step holds

    Therefore, factorial_recursive(n) correctly computes n!
    """
    if n == 0:
        return 1
    else:
        return n * factorial_recursive(n - 1)

# Verification
for n in range(0, 8):
    import math
    computed = factorial_recursive(n)
    expected = math.factorial(n)
    print(f"{n}! = {computed}, Expected = {expected}, Match = {computed ==
expected}")
```

Example 3: Proving Properties of Data Structures

```
def power_of_2_check(n):
    """
    Claim: All positive integers that are powers of 2 can be exactly
    divided by 2 until reaching 1.

    Proof by Mathematical Induction:

    Base Case (n=1=2^0):
    n = 1, which is 2^0
    ✓ Base case is a power of 2
```

```
    Inductive Step:
    Assume 2^k is a power of 2
    Prove 2^(k+1) is a power of 2

    2^(k+1) = 2 × 2^k
    Since 2^k is a power of 2 (by inductive hypothesis),
    2 × 2^k is also a power of 2
    ✓ Inductive step holds

    Therefore, 2^n is a power of 2 for all non-negative integers n
    """
    pass

def verify_power_of_2_divisibility(n):
    """
    Verify: Powers of 2 can be divided by 2 repeatedly until reaching 1
    """
    value = n
    iterations = 0

    while value > 1:
        if value % 2 != 0:
            return False, iterations
        value //= 2
        iterations += 1

    return True, iterations

# Verification
powers_of_2 = [1, 2, 4, 8, 16, 32, 64, 128, 256]
for n in powers_of_2:
    result, iterations = verify_power_of_2_divisibility(n)
    print(f"n={n}: Divisible by 2 repeatedly = {result}, Iterations = {iterations}")
```

3.6.3 Case Analysis

Case analysis proves correctness by considering all possible cases and showing the algorithm or property holds in each case. This technique is essential for proving algorithms that behave differently based on input characteristics.

Example 1: Maximum Element in Array

```
def find_maximum(arr):
    """
    Find maximum element in array

    Proof by Case Analysis:

    Case 1: Array has 1 element
    Maximum is that element ✓

    Case 2: Array has n elements (n > 1)
    We compare arr[0] with find_maximum(arr[1:])
    Subcase 2a: arr[0] >= find_maximum(arr[1:])
            Maximum is arr[0] ✓
    Subcase 2b: arr[0] < find_maximum(arr[1:])
            Maximum is find_maximum(arr[1:]) ✓
```

```
    All cases covered: Algorithm is correct for all inputs
    """
    if len(arr) == 1:
        return arr[0]
    else:
        return max(arr[0], find_maximum(arr[1:]))

# Testing - Verification of correctness through case analysis
test_arrays = [
    [5],                    # Case 1: Single element
    [3, 7, 2],              # Case 2a: Max at beginning
    [1, 2, 9, 7, 2],        # Case 2a: Max at middle
    [1, 2, 3, 4, 9],        # Case 2a: Max at end
    [-5, -10, -1, -3],      # All negative numbers
    [0, 0, 0, 0],           # All same elements
]

for arr in test_arrays:
    result = find_maximum(arr)
    expected = max(arr)
    print(f"Array: {arr}, Max: {result}, Expected: {expected}, Correct: {result ==
expected}")
```

Example 2: Binary Search Correctness through Case Analysis

```
def binary_search_with_cases(arr, target):
    """
    Binary Search using case analysis for correctness proof

    Case 1: Array is empty
    Target cannot be found → return -1

    Case 2: Array has one element
    Subcase 2a: arr[0] == target → found, return 0
    Subcase 2b: arr[0] != target → not found, return -1

    Case 3: Array has multiple elements
    Subcase 3a: target at mid → found, return mid
    Subcase 3b: target < arr[mid] → search left half (recursively covered)
    Subcase 3c: target > arr[mid] → search right half (recursively covered)

    All cases are correctly handled
    """
    if not arr:
        return -1

    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:  # Case 3a
            return mid
        elif arr[mid] < target:  # Case 3c
            left = mid + 1
        else:  # Case 3b
            right = mid - 1
```

```
    return -1

# Test cases covering all scenarios
test_cases = [
    ([], 5, -1),                        # Empty array
    ([5], 5, 0),                        # Single element - found
    ([5], 3, -1),                       # Single element - not found
    ([1, 3, 5, 7, 9], 5, 2),            # Target at middle
    ([1, 3, 5, 7, 9], 1, 0),            # Target at beginning
    ([1, 3, 5, 7, 9], 9, 4),            # Target at end
    ([1, 3, 5, 7, 9], 4, -1),           # Target not found
    ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 7, 6), # Larger array
]

for arr, target, expected in test_cases:
    result = binary_search_with_cases(arr, target)
    status = "✓" if result == expected else "✗"
    print(f"{status} Array: {arr}, Target: {target}, Result: {result}, Expected:
{expected}")
```

3.6.4 Contradiction Proof

Contradiction proof (also called proof by contradiction or reductio ad absurdum) assumes the negation of what we want to prove and shows this leads to an impossible situation or contradiction.

Example 1: Lower Bound for Comparison-Based Sorting

```
def sorting_lower_bound_analysis():
    """
    Theorem: Any comparison-based sorting algorithm requires at least
    Ω(n log n) comparisons in the worst case.

    Proof by Contradiction:

    Assume: There exists a comparison-based sorting algorithm A that
            performs fewer than n log n comparisons in worst case.

    Reasoning:
    1. Any comparison-based algorithm can be represented as a decision tree
       where each node represents a comparison and each leaf represents
       an output (permutation).

    2. For n elements, there are n! possible permutations (orderings)
       that must be distinguishable → need at least n! leaves

    3. A binary tree with k comparisons has at most 2^k leaves
       (each comparison has 2 outcomes: true or false)

    4. Therefore: 2^k ≥ n!
       Taking log: k ≥ log   (n!)

    5. By Stirling's approximation:
       log   (n!) ≈ n log   (n) - n log   (e) ≈ n log   (n)

    6. So: k ≥ n log   (n) = Θ(n log n)

    Contradiction: We assumed k < n log n, but proved k ≥ n log n
```

```
    Conclusion: No comparison-based sorting can do better than Θ(n log n)
    """
    import math

    # Demonstrate the inequality: 2^k ≥ n!
    for n in range(1, 13):
        factorial_n = math.factorial(n)
        min_comparisons_needed = math.log2(factorial_n)
        n_log_n = n * math.log2(n)

        print(f"n={n}: n! = {factorial_n:>8}, log   (n!) =
{min_comparisons_needed:>6.2f}, n log n = {n_log_n:>6.2f}")

sorting_lower_bound_analysis()
```

Output interpretation: The table shows that log (n!) is always approximately equal to n log n, confirming the Ω(n log n) lower bound.

3.7 SUMMARY

Key Takeaways

- Algorithm analysis separates efficient solutions from inefficient ones
- Asymptotic notation provides a language to discuss complexity independent of hardware
- For large inputs, choosing the right algorithm can mean the difference between seconds and years of computation
- The seven fundamental functions cover most practical algorithms
- Proving correctness is as important as analyzing efficiency
- Understanding algorithm analysis is essential for any software engineer

Algorithm analysis is the process of determining computational resource requirements of algorithms. It allows us to:
- Compare algorithms objectively
- Predict performance on large inputs
- Make informed design decisions
- Identify scalability bottlenecks

Aspect	Experimental	Theoretical
Method	Run actual code	Mathematical analysis
Hardware Dependent	Yes	No
Scalability	Limited	Unlimited
Implementation Sensitive	Yes	No
Practical Value	High	High

The Seven Functions:

These represent common complexity classes in algorithm analysis:

1. O(1) - Constant: Array access, arithmetic operations
2. O(log n) - Logarithmic: Binary search
3. O(n) - Linear: Simple loops, linear search
4. O(n log n) - Linear-logarithmic: Optimal sorting algorithms
5. O(n^2) - Quadratic: Nested loops, elementary sorting
6. O(n^3) - Cubic: Triple nested loops, matrix operations
7. O(2^n) - Exponential: Brute force, certain recursive problems

Asymptotic Notation:

- Big-O: Upper bound (worst-case scenario)
- Omega: Lower bound (best-case scenario)
- Theta: Tight bound (when upper and lower bounds match)

Justification Techniques:

- Loop Invariants: Prove correctness of iterative algorithms
- Mathematical Induction: Prove properties hold for all cases
- Case Analysis: Verify all possible scenarios
- Contradiction: Prove impossibility of alternatives

3.7.2 Decision Guide: Choosing Algorithms Based on Complexity

For $n \leq 10$: Any algorithm works, even $O(n^3)$ or $O(2^n)$
For $n \leq 1{,}000$: $O(n^2)$ algorithms are acceptable $O(n^3)$ becomes questionable $O(2^n)$ is infeasible
For $n \leq 1{,}000{,}000$: $O(n)$ and $O(n \log n)$ are required $O(n^2)$ starts to become problematic Anything worse is unacceptable
For $n > 1{,}000{,}000$: Only $O(1)$, $O(\log n)$, or $O(n)$ are acceptable $O(n \log n)$ is marginal Anything worse is completely infeasible

3.7.3 Common Analysis Patterns

```
# Pattern 1: Simple Loop - O(n)
for i in range(n):
    # O(1) operation

# Pattern 2: Nested Loop - O(n²)
for i in range(n):
    for j in range(n):
        # O(1) operation

# Pattern 3: Conditional Loop - O(n)
i = 1
while i <= n:
    # O(1) operation
    i *= 2  # Reduces effective iterations

# Pattern 4: Sequential Operations - O(n + m)
for i in range(n):
    # O(1) operation
for j in range(m):
    # O(1) operation

# Pattern 5: Divide and Conquer - O(n log n)
def divide_conquer(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = divide_conquer(arr[:mid])
    right = divide_conquer(arr[mid:])
    return merge(left, right)  # O(n) merge
```

Check Your Progress

1) Determine the time complexity of the following function:

```
def mystery_function(arr):
    count = 0
    for i in range(len(arr)):
        for j in range(len(arr)):
            if arr[i] == arr[j]:
                count += 1
    return count
```

2) Determine the time complexity of the following function:

```
def another_function(arr):
    total = 0
    for i in range(len(arr)):
        total += arr[i]
    for i in range(len(arr)):
        total += arr[i] * 2
    return total
```

3) Compare bubble sort $O(n^2)$ with merge sort $O(n \log n)$ for sorting 1 million elements.

```
# Solution with calculations
n = 1_000_000

bubble_sort_ops = n  2
merge_sort_ops = n * import_math.log2(n)

print(f"Bubble Sort Operations: {bubble_sort_ops:,}")
print(f"Merge Sort Operations: {merge_sort_ops:,.0f}")
print(f"Bubble Sort is {bubble_sort_ops / merge_sort_ops:,.0f}x slower")

# Output approximately:
# Bubble Sort Operations: 1,000,000,000,000
# Merge Sort Operations: 19,931,568
# Bubble Sort is 50,178x slower
```

Answer: Merge sort is approximately 50,000 times faster for 1 million elements.

4) Prove by mathematical induction that the sum of first **n** squares equals **n(n+1)(2n+1)/6**.

5) Analyze the time complexity of the following recursive function:

```
def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)
```

6) A company needs to search through 100,000 customer records. Would you recommend linear search or binary search? Justify your answer.

7) A company needs to search through 100,000 customer records. Would you recommend linear search or binary search? Justify your answer.

3.8 SOLUTIONS / ANSWERS

1)
Solution:

- Outer loop: n iterations
- Inner loop: n iterations (for each outer iteration)
- Inside inner loop: O(1) comparison
- Total: $n \times n = O(n^2)$

Answer: $O(n^2)$ - Quadratic Time Complexity

2)
Solution:

- First loop: n iterations, O(n)
- Second loop: n iterations, O(n)
- Sequential operations add: O(n) + O(n) = O(2n) = O(n)
- Total: O(n)

Answer: O(n) - Linear Time Complexity

3)

```
# Solution with calculations
n = 1_000_000

bubble_sort_ops = n  2
merge_sort_ops = n * import_math.log2(n)

print(f"Bubble Sort Operations: {bubble_sort_ops:,}")
print(f"Merge Sort Operations: {merge_sort_ops:,.0f}")
print(f"Bubble Sort is {bubble_sort_ops / merge_sort_ops:,.0f}x slower")

# Output approximately:
# Bubble Sort Operations: 1,000,000,000,000
# Merge Sort Operations: 19,931,568
# Bubble Sort is 50,178x slower
```

Answer: Merge sort is approximately 50,000 times faster for 1 million elements.

4)
Solution:

- a = 4, b = 2, f(n) = n
- n^(log_b a) = n^(log 4) = n^2
- Compare f(n) with n^2:
  - f(n) = n = O(n^(2-ε)) for ε = 1
  - Since f(n) is smaller than n^(log_b a), Case 1 applies
- $T(n) = \Theta(n^2)$

Answer: $T(n) = \Theta(n^2)$ - Quadratic Time Complexity

5)
Solution:

```
def sum_of_squares_formula(n):
    """
    Prove: 1² + 2² + 3² + ... + n² = n(n+1)(2n+1)/6

    Proof by Mathematical Induction:

    Base Case (n=1):
    Left side: 1² = 1
    Right side: 1(1+1)(2+1)/6 = 1×2×3/6 = 1
    ✓ Base case holds
```

```
    Inductive Step:
    Assume: 1² + 2² + ... + k² = k(k+1)(2k+1)/6
    Prove:  1² + 2² + ... + k² + (k+1)² = (k+1)(k+2)(2k+3)/6

    LHS = k(k+1)(2k+1)/6 + (k+1)²
        = [k(k+1)(2k+1) + 6(k+1)²]/6
        = [(k+1)(k(2k+1) + 6(k+1))]/6
        = [(k+1)(2k² + k + 6k + 6)]/6
        = [(k+1)(2k² + 7k + 6)]/6
        = [(k+1)(k+2)(2k+3)]/6
        = RHS
    ✓ Inductive step holds
    """
    return n * (n + 1) * (2 * n + 1) // 6

# Verification
for n in range(1, 11):
    direct_sum = sum(i2 for i in range(1, n + 1))
    formula_result = sum_of_squares_formula(n)
    print(f"n={n}: Sum={direct_sum}, Formula={formula_result}, Match={direct_sum
== formula_result}")
```

6)

Solution:

```
def analyze_fibonacci_recursive():
    """
    Recurrence Relation: T(n) = T(n-1) + T(n-2) + O(1)

    Expanding the recurrence:
    T(n) = T(n-1) + T(n-2)
        = [T(n-2) + T(n-3)] + T(n-2)
        = 2T(n-2) + T(n-3)
        ...

    This forms a pattern similar to Fibonacci sequence itself
    Counting function calls:
    - Level 0: 1 call to fib(n)
    - Level 1: 2 calls (fib(n-1), fib(n-2))
    - Level 2: 3 calls (fib(n-2) appears twice, fib(n-3))
    - ...
    - Total calls ≈ φ^n where φ = (1+√5)/2 ≈ 1.618

    Therefore: T(n) = Θ(φ^n) ≈ Θ(1.618^n)

    This is exponential - extremely inefficient!
    For n=40: ~330 million function calls
    For n=50: ~40 billion function calls
    """
    pass
```

Answer: T(n) = Θ(1.618^n) - Exponential Time Complexity

7)

Solution:

```
# Analysis
n = 100_000

# Linear Search: O(n)
linear_worst_case = n
```

```
linear_average = n / 2

# Binary Search: O(log n) - requires sorted data
binary_worst_case = import_math.log2(n)

print(f"Linear Search Worst Case: {linear_worst_case:,} comparisons")
print(f"Linear Search Average Case: {linear_average:,} comparisons")
print(f"Binary Search Worst Case: {binary_worst_case:.0f} comparisons")
print(f"Speedup: {linear_average / binary_worst_case:.0f}x faster")
```

Answer: Recommend binary search because:

1. Binary search is ~1,500x faster in the worst case
2. Slight overhead of sorting (done once) is negligible
3. For repeated searches, binary search becomes increasingly advantageous
4. Even if data needs sorting: Sort O(n log n) + Search O(log n) = O(n log n) total
5. This is better than repeated O(n) linear searches

3.9 FURTHER READINGS

1. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** *Introduction to Algorithms* (3rd ed.). MIT Press.
    - Comprehensive coverage of algorithm analysis and design
    - Rigorous mathematical treatment of complexity classes
2. **Goodrich, M. T., & Tamassia, R. (2014).** *Algorithm Design Manual* (2nd ed.). Wiley.
    - Practical approach to algorithm analysis
    - Extensive examples and case studies
3. **Kleinberg, J., & Tardos, É. (2005).** *Algorithm Design*. Pearson.
    - Clear explanations of complexity analysis
    - Excellent treatment of asymptotic notation

Online Resources

1. **Khan Academy - Algorithms**: Free courses on algorithm analysis
    - Interactive visualizations of algorithm complexity
    - Video explanations of key concepts
2. **MIT OpenCourseWare - Introduction to Algorithms**: Full course materials
    - Lectures from MIT's legendary algorithms course
    - Problem sets and solutions
3. **Visualization Tools**:
    - VisuAlgo.net: Visualize algorithm execution and complexity
    - Big-O Cheat Sheet: Quick reference for common complexities

Practice Problems

1. **LeetCode.com**: 2,000+ algorithm problems with solutions
2. **HackerRank.com**: Interactive coding challenges with complexity analysis
3. **ProjectEuler.net**: Mathematical and computational problems

UNIT 4 RECURSION

Structure

4.0 Introduction
4.1 Objectives
4.2 Illustrative Examples
4.3 Analyzing Recursive Algorithms
4.4 Recursion Run Amok
4.5 Further Examples of Recursion
4.6 Designing Recursive Algorithms
4.7 Eliminating Tail Recursion
4.8 Summary
4.9 Solutions/Answers
4.10 Further Readings

4.0 INTRODUCTION

Recursion is one of the most powerful and elegant problem-solving techniques in computer science and programming. At its core, recursion is a method of solving problems by breaking them down into smaller, similar subproblems. A recursive solution involves a function calling itself with modified parameters until it reaches a base case that can be solved directly.

The concept of recursion appears naturally in many areas of mathematics and computer science. From calculating factorials and Fibonacci sequences to traversing tree structures and solving complex algorithmic problems, recursion provides an intuitive way to express solutions that would otherwise require complex iterative logic.

The following are key characteristics of Recursion:

- **Self-reference**: A function calls itself
- **Problem decomposition**: Large problems are broken into smaller, similar problems
- **Base case**: A condition that stops the recursive calls
- **Recursive case**: The part where the function calls itself with modified parameters

Understanding recursion is crucial for mastering advanced data structures like trees and graphs, as well as algorithmic techniques such as divide-and-conquer, dynamic programming, and backtracking.

4.1 OBJECTIVES

After studying this unit, you should be able to :

- Define and explain the concept of recursion and its fundamental principles
- Identify the essential components of a recursive algorithm (base case and recursive case)
- Trace through recursive function calls using stack diagrams and execution trees
- Analyze the time and space complexity of recursive algorithms
- Recognize when recursion can lead to inefficient solutions (recursion run amok)
- Design recursive algorithms for various problem domains

- Convert recursive solutions to iterative ones and understand tail recursion elimination
- Apply recursion to solve real-world problems including mathematical computations, data structure operations, and algorithmic challenges
- Debug recursive functions and avoid common pitfalls
- Compare recursive and iterative approaches to determine the most appropriate solution method

4.2 ILLUSTRATIVE EXAMPLES

The following are some of the examples of Recursion:

4.2.1 Mathematical Recursion: Factorial

The factorial function is the classic introduction to recursion. The factorial of a non-negative integer n (denoted as n!) is the product of all positive integers less than or equal to n.

Mathematical Definition:

- 0! = 1 (base case)
- n! = n × (n-1)! for n > 0 (recursive case)

Recursive Implementation:

```
def factorial(n):
    # Base case
    if n == 0 or n == 1:
        return 1
    # Recursive case
    else:
        return n * factorial(n - 1)

# Example usage
print(factorial(5))  # Output: 120
print(factorial(0))  # Output: 1
```

Execution Trace for factorial(4):

```
factorial(4)
  └─ 4 * factorial(3)
      └─ 4 * 3 * factorial(2)
          └─ 4 * 3 * 2 * factorial(1)
              └─ 4 * 3 * 2 * 1
              └─ 4 * 3 * 2
          └─ 4 * 6
      └─ 24
```

4.2.2 Fibonacci Sequence

The Fibonacci sequence is another classic example where each number is the sum of the two preceding numbers.

Mathematical Definition:

- F(0) = 0, F(1) = 1 (base cases)
- F(n) = F(n-1) + F(n-2) for n > 1 (recursive case)

Recursive Implementation:

```
def fibonacci(n):
    # Base cases
    if n == 0:
        return 0
    elif n == 1:
```

```
        return 1
    # Recursive case
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

# Example usage
print(fibonacci(6))  # Output: 8
print(fibonacci(10)) # Output: 55
```

4.2.3 String Operations: Palindrome Check

```
def is_palindrome(s):
    # Base cases
    if len(s) <= 1:
        return True
    # Recursive case
    if s[0] == s[-1]:
        return is_palindrome(s[1:-1])
    else:
        return False

# Example usage
print(is_palindrome("racecar"))  # Output: True
print(is_palindrome("hello"))    # Output: False
```

4.2.4 Array Operations: Sum of Elements

```
def array_sum(arr, n):
    # Base case
    if n <= 0:
        return 0
    # Recursive case
    return array_sum(arr, n - 1) + arr[n - 1]

# Example usage
numbers = [1, 2, 3, 4, 5]
print(array_sum(numbers, len(numbers)))  # Output: 15
```

4.3 ANALYZING RECURSIVE ALGORITHMS

This section illustrates the process of analyzing recursive algorithms by example.

4.3.1 Call Stack Visualization

When a recursive function is called, each function call is placed on the call stack. Understanding this stack behavior is crucial for analyzing recursive algorithms.

Stack Diagram for factorial(4):

```
| factorial(1) | <- Top of stack (returns 1)
| factorial(2) |
| factorial(3) |
| factorial(4) | <- Bottom of stack (original call)
|_____________|
```

4.3.2 Time Complexity Analysis

Linear Recursion (Factorial):

- $T(n) = T(n-1) + O(1)$
- $T(1) = O(1)$
- Solution: $T(n) = O(n)$

Binary Recursion (Fibonacci - Naive):

- $T(n) = T(n-1) + T(n-2) + O(1)$
- $T(0) = T(1) = O(1)$
- Solution: $T(n) = O(\varphi^n)$ where $\varphi \approx 1.618$ (golden ratio)

4.3.3 Space Complexity Analysis

Space complexity is determined by the maximum depth of recursion (call stack depth):

- **Factorial**: O(n) space due to n recursive calls on the stack
- **Fibonacci (naive)**: O(n) space due to maximum recursion depth
- **Binary tree traversal**: O(h) space where h is the height of the tree

4.3.4 Recurrence Relations

Master Theorem for analyzing divide-and-conquer recurrences: For $T(n) = aT(n/b) + f(n)$:

1. If $f(n) = O(n^{(\log_b(a)-\varepsilon)})$ for $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b(a)})$
2. If $f(n) = \Theta(n^{\log_b(a)})$, then $T(n) = \Theta(n^{\log_b(a)} \times \log n)$
3. If $f(n) = \Omega(n^{(\log_b(a)+\varepsilon)})$ for $\varepsilon > 0$, then $T(n) = \Theta(f(n))$

4.4 RECURSION RUN AMOK

4.4.1 Exponential Time Complexity

The naive Fibonacci implementation demonstrates how recursion can lead to extremely inefficient solutions:

```
def fibonacci_naive(n):
    if n <= 1:
        return n
    return fibonacci_naive(n-1) + fibonacci_naive(n-2)
```

Problems with this approach:

- Time complexity: $O(2^n)$
- Many subproblems are solved multiple times
- For fibonacci_naive(40), approximately 2^{40} function calls are made

Call tree for fibonacci(5):

```
                fib(5)
              /       \
           fib(4)       fib(3)
          /    \       /    \
       fib(3)   fib(2)  fib(2)  fib(1)
      /   \    /   \   /   \
   fib(2) fib(1) fib(1) fib(0) fib(1) fib(0)
  /   \
fib(1) fib(0)
```

4.4.2 Stack Overflow

Deep recursion can lead to stack overflow errors:

```
def infinite_recursion(n):
    print(f"Call {n}")
```

```
    return infinite_recursion(n + 1)  # No base case!

# This will cause a stack overflow
# infinite_recursion(1)
```

4.4.3 Optimizing Recursive Solutions

Memoization - storing previously computed results:

```
def fibonacci_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fibonacci_memo(n-1, memo) + fibonacci_memo(n-2, memo)
    return memo[n]
```

Time complexity: O(n), Space complexity: O(n)

Dynamic Programming - bottom-up approach:

```
def fibonacci_dp(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

4.5 FURTHER EXAMPLES OF RECURSION

The following are some more examples of Recursion:

4.5.1 Tree Traversals

Binary Tree Structure:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Inorder Traversal (Left, Root, Right):

Algorithm Steps:

1. Check if the current node is empty (None)
2. If empty, return an empty list (base case)
3. Otherwise, create an empty result list
4. Recursively traverse the left subtree and add its values to result
5. Add the current node's value to result
6. Recursively traverse the right subtree and add its values to result
7. Return the complete result list

Inorder Traversal (Left, Root, Right):

```
def inorder_traversal(root):
    if root is None:
        return []
    result = []
```

```
    result.extend(inorder_traversal(root.left))
    result.append(root.val)
    result.extend(inorder_traversal(root.right))
    return result
```

Preorder Algorithm Steps:

1. Check if the current node is empty (None)
2. If empty, return an empty list (base case)
3. Otherwise, create a result list starting with the current node's value
4. Recursively traverse the left subtree and add its values to result
5. Recursively traverse the right subtree and add its values to result
6. Return the complete result list

Preorder Traversal (Root, Left, Right):

```
def preorder_traversal(root):
    if root is None:
        return []
    result = [root.val]
    result.extend(preorder_traversal(root.left))
    result.extend(preorder_traversal(root.right))
    return result
```

4.5.2 Divide and Conquer: Merge Sort

Merge Sort Algorithm Steps:

1. Check if array has 1 or fewer elements (base case - already sorted)
2. If base case, return the array as is
3. Otherwise, divide the array into two halves at the middle point
4. Recursively sort the left half
5. Recursively sort the right half
6. Merge the two sorted halves back together
7. Return the merged, sorted array

Merge Function Algorithm Steps:

1. Create an empty result array and two pointers for left and right arrays
2. Compare elements at current pointers in both arrays
3. Add the smaller element to result and advance its pointer
4. Repeat until one array is exhausted
5. Add any remaining elements from the non-empty array to result
6. Return the merged result

```
def merge_sort(arr):
    # Base case
    if len(arr) <= 1:
        return arr

    # Divide
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # Conquer
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
```

```
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Example usage
numbers = [64, 34, 25, 12, 22, 11, 90]
sorted_numbers = merge_sort(numbers)
print(sorted_numbers)  # Output: [11, 12, 22, 25, 34, 64, 90]
```

4.5.3 Backtracking: N-Queens Problem

N-Queens Problem Algorithm Steps:

1. Initialize an empty solution list to store all valid arrangements
2. Start placing queens row by row from the first row
3. For each row, try placing a queen in each column
4. Before placing a queen, check if the position is safe (no conflicts)
5. If safe, place the queen and recursively try to place queens in remaining rows
6. If all queens are placed successfully, add the solution to the list
7. If placement leads to no solution, backtrack and try the next position
8. Continue until all possibilities are explored

Safety Check Algorithm Steps:

1. Check if any queen in previous rows is in the same column
2. Check if any queen is on the same diagonal (both directions)
3. A position is safe only if no conflicts exist with previously placed queens

```
def solve_n_queens(n):
    def is_safe(board, row, col):
        # Check column
        for i in range(row):
            if board[i] == col:
                return False

        # Check diagonals
        for i in range(row):
            if abs(board[i] - col) == abs(i - row):
                return False

        return True

    def backtrack(board, row):
        if row == n:
            solutions.append(board[:])
            return

        for col in range(n):
            if is_safe(board, row, col):
                board[row] = col
                backtrack(board, row + 1)
                # No need to reset as we overwrite in the next iteration
```

```
    solutions = []
    backtrack([-1] * n, 0)
    return solutions

# Find all solutions for 4-Queens
solutions = solve_n_queens(4)
print(f"Number of solutions for 4-Queens: {len(solutions)}")
```

4.5.4 Graph Traversal: Depth-First Search

Depth-First Search (DFS) Algorithm Steps:

1. Initialize a set to keep track of visited vertices (avoid infinite loops)
2. Mark the starting vertex as visited
3. Process the current vertex (print, store, or perform required operation)
4. For each unvisited neighbor of the current vertex: a. Recursively call DFS on that neighbor b. The recursion will explore as deep as possible before backtracking
5. When no more unvisited neighbors exist, the function returns (backtracking)
6. The algorithm continues until all reachable vertices are visited

```
def dfs_recursive(graph, vertex, visited=None):
    if visited is None:
        visited = set()

    visited.add(vertex)
    print(vertex, end=' ')

    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)

    return visited

# Example graph representation (adjacency list)
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

dfs_recursive(graph, 'A')  # Output: A B D E F C
```

4.6 DESIGNING RECURSIVE ALGORITHMS

In this section, the process of designing recursive algorithms shall be discussed:

4.6.1 The Recursive Design Process

Step 1: Identify the Base Case(s)
Algorithm for Finding Base Cases:

1. Ask the Fundamental Question: "What is the simplest version of this problem that I can solve directly?"
2. Consider Edge Cases: Think about scenarios where:

    - Input size is 0 or 1
    - Range becomes empty or invalid
    - Target condition is already met
3. Ensure Termination: The base case must guarantee that recursion stops
4. Provide Direct Solution: Return an immediate answer without further recursion
5. Examples of Common Base Cases:
    - Empty collection: return default value (0 for sum, 1 for product, empty list for concatenation)
    - Single element: return that element or simple transformation
    - Invalid range: return appropriate error value or empty result
    - Goal achieved: return success indicator or accumulated result

Step 2: Define the Recursive Case

Algorithm for Designing Recursive Case:

1. Problem Decomposition Strategy:
    - Break the problem into smaller, similar subproblems
    - Each subproblem should be of the same type as the original
    - Subproblems should be simpler (smaller size, reduced range, etc.)
2. Choose Decomposition Method:
    - Linear: Process one element, recurse on the rest ($n \rightarrow n-1$)
    - Binary: Split in half, recurse on both parts ($n \rightarrow n/2 + n/2$)
    - Multiple: Split into several parts based on problem structure
3. Define Combination Logic:
    - How do you merge results from subproblems?
    - Examples: addition, multiplication, concatenation, selection of optimal result

Step 3: Ensure Progress Toward Base Case

Algorithm for Ensuring Progress:

1. Parameter Modification Check:
    - Each recursive call must change parameters in a meaningful way
    - Changes should consistently move toward base case conditions
2. Convergence Verification:
    - Trace through a few examples manually
    - Verify that parameter values approach base case conditions
    - Ensure no infinite loops are possible
3. Common Progress Patterns:
    - Size reduction: $n \rightarrow n-1$, $n \rightarrow n/2$
    - Range narrowing: [start, end] → [start, mid] and [mid+1, end]
    - State advancement: moving through data structure, updating counters

Complete Design Process Example:

Problem: Calculate sum of array elements

- Step 1 - Base Case:
  - Question: "What's the simplest array sum?"
  - Answer: Empty array sums to 0
  - Base Case: if len(arr) == 0, return 0

  Step 2 - Recursive Case:
  - Decomposition: First element + sum of remaining elements
  - Recursive Call: sum_array(arr[1:])
  - Combination: arr[0] + recursive_result

  Step 3 - Progress Check:
  - Parameter Change: Array size decreases by 1 each call
  - Convergence: Eventually reaches empty array
  - No infinite loops possible

4.6.2 Design Patterns

Linear Recursion Pattern:
Linear Recursion Algorithm Steps:

1. Base Case Check: Determine if the problem size has reached the simplest form
    - Usually when size equals 0, 1, or some minimal condition
    - Return the direct solution for this base case
2. Problem Decomposition: Break the problem into:
    - Current element or operation
    - Remaining subproblem of size (n-1)
3. Recursive Call: Make a single recursive call with reduced problem size
    - Process one element and pass the rest to recursion
    - Trust that recursion will solve the smaller problem correctly
4. Combine Solutions: Merge the current element's contribution with the recursive result
    - This step varies based on the specific problem (sum, product, concatenation, etc.)

```
def linear_recursive_template(problem, size):
    # Base case
    if size == base_condition:
        return base_solution

    # Recursive case
    subproblem_solution = linear_recursive_template(problem, size - 1)
    return combine(current_element, subproblem_solution)
```

Binary Recursion Pattern:
Binary Recursion Algorithm Steps:

1. Base Case Check: Verify if the range is invalid or minimal
    - Usually when start >= end or size <= threshold
    - Return appropriate base solution
2. Divide Step: Split the problem into two roughly equal parts
    - Calculate midpoint to divide the range
    - Create two subproblems of approximately half the size
3. Conquer Step: Recursively solve both subproblems
    - Make two recursive calls for left and right halves
    - Each call handles its respective portion independently
4. Combine Step: Merge solutions from both subproblems
    - The combination method depends on the problem type
    - Examples: merge sorted arrays, sum values, find maximum, etc.
6. Key Advantage: Achieves O(log n) depth in recursion tree, leading to better performance for many problems.

```
def binary_recursive_template(problem, start, end):
    # Base case
    if start >= end:
        return base_solution

    # Divide
    mid = (start + end) // 2
    left_solution = binary_recursive_template(problem, start, mid)
    right_solution = binary_recursive_template(problem, mid + 1, end)

    # Conquer
    return combine(left_solution, right_solution)
```

4.6.3 Example: Tower of Hanoi

Problem Understanding: The Tower of Hanoi is a classic puzzle where you must move all disks from one peg to another, following these rules:

- Only one disk can be moved at a time
- A larger disk cannot be placed on top of a smaller disk
- You can use a third peg as auxiliary storage
- Tower of Hanoi Algorithm Steps:

Base Case Check: If there's only 1 disk to move:

  - Simply move it directly from source to destination
  - Print the move and return (no further recursion needed)

Recursive Case (3 steps for n > 1):

  - Step 1: Move the top (n-1) smaller disks from source to auxiliary peg
    - Use destination peg as temporary storage during this process
    - This clears the way for the largest disk
  - Step 2: Move the largest disk from source to destination
    - Now it's safe to move since all smaller disks are on auxiliary peg
    - Print this move
  - Step 3: Move the (n-1) disks from auxiliary to destination
    - Place them on top of the largest disk
    - Use source peg as temporary storage during this process

Recursive Structure: Each recursive call solves a smaller version of the same problem, ensuring progress toward the base case.

```
def tower_of_hanoi(n, source, destination, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    # Move n-1 disks from source to auxiliary
    tower_of_hanoi(n-1, source, auxiliary, destination)

    # Move the largest disk from source to destination
    print(f"Move disk {n} from {source} to {destination}")

    # Move n-1 disks from auxiliary to destination
    tower_of_hanoi(n-1, auxiliary, destination, source)

# Example usage
tower_of_hanoi(3, 'A', 'C', 'B')
```

4.6.4 Example: Generate All Subsets

Problem Understanding: Generate all possible subsets (power set) of a given set. For a set with n elements, there are 2^n possible subsets, including the empty set and the set itself.

Generate All Subsets Algorithm Steps:

Main Function:

1. Initialize an empty result list to store all subsets
2. Call the backtracking helper function starting with:
   - Start index = 0 (begin with first element)
   - Empty current subset list
3. Return the complete list of all generated subsets

Backtracking Helper Function Steps:

1. Add Current State: Add a copy of the current subset to the result list
   - This represents one valid subset at this point in exploration
   - Use a copy to avoid reference issues when backtracking

2. Explore Further Choices: For each remaining element from the start index:
    - Make Choice: Add the current element to the current subset
    - Recurse: Explore all subsets that include this element
        - Increment start index to avoid duplicate combinations
        - Pass the modified current subset
    - Backtrack: Remove the element from current subset
        - This undoes the choice and prepares for the next iteration
        - Essential for exploring subsets without this element
3. Implicit Base Case: When start index reaches the end of array:
    - No more elements to consider
    - Function returns naturally (backtracking occurs)

Key Insights:

- Each element has 2 choices: include it or exclude it
- The algorithm systematically explores both choices for every element
- Backtracking ensures all combinations are generated without duplication

```
def generate_subsets(nums):
    def backtrack(start, current_subset):
        # Add current subset to result
        result.append(current_subset[:])

        # Generate subsets by including each remaining element
        for i in range(start, len(nums)):
            current_subset.append(nums[i])
            backtrack(i + 1, current_subset)
            current_subset.pop()  # Backtrack

    result = []
    backtrack(0, [])
    return result

# Example usage
subsets = generate_subsets([1, 2, 3])
print(subsets)  # Output: [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
```

4.7 ELIMINATING TAIL RECURSION

4.7.1 Understanding Tail Recursion

Tail recursion occurs when the recursive call is the last operation in the function. This is significant because tail-recursive functions can be optimized by compilers to use constant stack space.

Tail Recursive Example:

```
def factorial_tail_recursive(n, accumulator=1):
    # Base case
    if n == 0 or n == 1:
        return accumulator
    # Tail recursive case
    return factorial_tail_recursive(n - 1, n * accumulator)
```

Non-Tail Recursive (Original):

```
def factorial_non_tail(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_non_tail(n - 1)  # Multiplication happens after recursive call
```

4.7.2 Converting to Iterative Solutions

Tail Recursive to Iterative:

```
def factorial_iterative(n):
    accumulator = 1
    while n > 1:
        accumulator *= n
        n -= 1
    return accumulator
```

Fibonacci - Tail Recursive:

```
def fibonacci_tail_recursive(n, a=0, b=1):
    if n == 0:
        return a
    return fibonacci_tail_recursive(n - 1, b, a + b)
```

Fibonacci - Iterative:

```
def fibonacci_iterative(n):
    if n <= 1:
        return n

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

4.7.3 Using Explicit Stack for General Recursion

For non-tail recursive functions, we can simulate recursion using an explicit stack:

Tree Traversal - Iterative with Stack:

```
def inorder_iterative(root):
    stack = []
    result = []
    current = root

    while stack or current:
        # Go to the leftmost node
        while current:
            stack.append(current)
            current = current.left

        # Current must be None here
        current = stack.pop()
        result.append(current.val)

        # Visit right subtree
        current = current.right

    return result
```

4.7.4 Benefits and Trade-offs

Advantages of Eliminating Recursion:

- **Constant space complexity** for tail recursion
- **Avoid stack overflow** for large inputs
- **Better performance** in some cases

Disadvantages:

- **Code complexity** may increase
- **Loss of elegance** and readability

- **Not always straightforward** to convert

4.8 SUMMARY

Recursion is a fundamental programming technique that provides elegant solutions to many complex problems. In this unit, the following are discussed:

- Recursion involves a function calling itself with modified parameters
- Every recursive function must have base cases to terminate recursion
- The call stack manages recursive function calls
- Recursion can lead to exponential time complexity if not carefully designed

The following are some of the analysis techniques:

- Time complexity analysis using recurrence relations
- Space complexity determined by recursion depth
- Master theorem for divide-and-conquer algorithms
- Recognition of when recursion becomes inefficient

The following are some of the optimization strategies:

- Memoization to avoid redundant calculations
- Tail recursion for space efficiency
- Converting recursive solutions to iterative ones
- Using explicit stacks when necessary

The following are some of the application domains:

- Mathematical computations (factorial, Fibonacci)
- Tree and graph algorithms
- Divide-and-conquer algorithms
- Backtracking problems
- String and array processing

The following are some of the best practices:

- Always define clear base cases
- Ensure progress toward base cases
- Consider iterative alternatives for simple linear recursions
- Use memoization for overlapping subproblems
- Be aware of stack overflow risks

The following guides on *when to use recursion*:

- Problem has recursive structure (trees, fractals)
- Divide-and-conquer approach is natural
- Backtracking is required
- Code clarity is significantly improved

The following guides on *when to avoid recursion* :

- Simple iterative solution exists
- Stack overflow is a concern
- Performance is critical and recursion adds overhead
- Tail recursion can be easily converted to iteration

Check Your Progress

1) Trace the execution of factorial(5).
2) Write a recursive function to find the maximum element in an array.
3) Calculate the time complexity of Binary Search Algorithm.
4) Convert the following recursive function to iterative function:

```
def sum_digits_recursive(n):
    if n == 0:
        return 0
    return n % 10 + sum_digits_recursive(n // 10)
```

5) Implement recursive Binary Search algorithm.

4.9SOLUTIONS/ANSWERS

1)**Solution:**

```
factorial(5)
├─ 5 * factorial(4)
  ├─ 5 * 4 * factorial(3)
    ├─ 5 * 4 * 3 * factorial(2)
      ├─ 5 * 4 * 3 * 2 * factorial(1)
        └─ 5 * 4 * 3 * 2 * 1 = 120
```

2)**Solution:**

```
def find_max_recursive(arr, n):
    # Base case
    if n == 1:
        return arr[0]

    # Recursive case
    max_of_rest = find_max_recursive(arr, n - 1)
    return max(arr[n - 1], max_of_rest)
```

3)Solution:

- Recurrence: T(n) = T(n/2) + O(1)
- Using Master Theorem: a=1, b=2, f(n)=O(1)
- Since f(n) = O(n^0) = O(1) and log (1) = 0
- Time complexity: T(n) = O(log n)

4)Solution:

```
def sum_digits_iterative(n):
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    return total
```

5)**Solution:**

```
def binary_search_recursive(arr, target, left=0, right=None):
    if right is None:
        right = len(arr) - 1

    # Base case
    if left > right:
        return -1

    mid = (left + right) // 2
```

```
    if arr[mid] == target:
        return mid
    elif arr[mid] > target:
        return binary_search_recursive(arr, target, left, mid - 1)
    else:
        return binary_search_recursive(arr, target, mid + 1, right)
```

4.10 FURTHER READINGS

1. "Introduction to Algorithms" by Cormen, Leiserson, Rivest, and Stein (CLRS)
    - Comprehensive coverage of recursive algorithms and analysis
    - Chapter 4: Divide-and-Conquer
2. "The Algorithm Design Manual" by Steven S. Skiena
    - Practical approach to algorithm design
    - Extensive coverage of recursive problem-solving
3. "Algorithms" by Robert Sedgewick and Kevin Wayne
    - Clear explanations of recursive algorithms
    - Focus on implementation and analysis
4. "Programming Pearls" by Jon Bentley
    - Insights into elegant recursive solutions
    - Problem-solving techniques

Online Resources

1. MIT OpenCourseWare - Introduction to Algorithms
    - Lecture notes and videos on recursive algorithms
2. Stanford CS106B Programming Abstractions
    - Comprehensive coverage of recursion with examples
3. Visualizing Recursion - Online visualization tools
    - Interactive demonstrations of recursive algorithms
4. LeetCode and HackerRank
    - Practice problems involving recursion

Practice Problems

1. Project Euler - Mathematical problems requiring recursive solutions
2. Codeforces - Competitive programming problems
3. AtCoder - Algorithm contests with recursive challenges
4. USACO - Programming contest problems

Specialized Topics for Further Study

1. Mutual Recursion - Functions calling each other recursively
2. Continuation-Passing Style - Advanced recursive programming technique
3. Recursive Data Structures - Self-referential data structures
4. Functional Programming - Recursion as the primary control structure
5. Logic Programming - Recursion in Prolog and similar languages

ignou THE PEOPLE'S UNIVERSITY